

Avaliacao 02 — Parte II — Processo ETL

Modelagem de Data Warehouse — EEL890 (UFRJ)

- 1 Capa
 - 1.1 Grupo
- 2 1. Introdução
 - 2.1 1.1 Objetivo do ETL
 - 2.2 1.2 Escopo do documento
- 3 2. Arquitetura do ETL
 - 3.1 2.1 Diagrama em texto
 - 3.2 2.2 Tempos de acionamento — suposição operacional
 - 3.3 2.3 Por que PostgreSQL único com schemas separados
- 4 3. Fontes consumidas e tradução MySQL → PostgreSQL
 - 4.1 3.1 As 5 fontes integradas
 - 4.2 3.2 Tradução MySQL → PostgreSQL
 - 4.3 3.3 Justificativa: por que ANSI SQL:1999+
- 5 4. Grupos excluídos e motivos
 - 5.1 4.1 EEL890-big-data (excluída)
 - 5.2 4.2 EEL890---Big-Data (excluída)
 - 5.3 4.3 locadora-oltp (excluída)
 - 5.4 4.4 Resumo
- 6 5. Etapas do ETL
 - 6.1 5.1 Extract — 5 scripts (etl/01_extract_*.sql a etl/05_extract_*.sql)
 - 6.1.1 5.1.1 Extract andre_gustavo (sk_fonte=1)
 - 6.1.2 5.1.2 Extract mae016 (sk_fonte=2)
 - 6.1.3 5.1.3 Extract locadora_db (sk_fonte=3)
 - 6.1.4 5.1.4 Extract bd_dw_26_1 (sk_fonte=4)
 - 6.1.5 5.1.5 Extract bigdata (sk_fonte=5)
 - 6.2 5.2 Transform — etl/06_transform.sql
 - 6.3 5.3 Load — etl/07_load_dimensoes.sql + etl/08_load_fatos.sql
 - 6.3.1 5.3.1 Load de dimensões (07_load_dimensoes.sql)
 - 6.3.2 5.3.2 Load de fatos (08_load_fatos.sql)
 - 6.3.3 5.3.3 Sentinelas e regras de NULL

7 6. Relatórios e Matriz de Markov

7.1 6.1 Relatório (a) — Controle de pátio (relatorios/
01_controle_patio.sql)

7.2 6.2 Relatório (b) — Controle das locações (relatorios/
02_controle_locacoes.sql)

7.3 6.3 Relatório (c) — Controle de reservas (relatorios/
03_controle_reservas.sql)

7.4 6.4 Relatório (d) — Grupos mais alugados (relatorios/
04_grupos_mais_alugados.sql)

7.5 6.5 Matriz de Markov (relatorios/05_matriz_markov.sql)

8 7. Problemas encontrados durante o desenvolvimento

8.1 7.1 CRÍTICO — Relatório (a) sem corte por marca/modelo/
mecanização (dimensional)

8.2 7.2 CRÍTICO — Grão de fato_patio_diario contraditório (dimen-
sional)

8.3 7.3 CRÍTICO — locadora_db sem amarração veiculo → pátio (di-
mensional)

8.4 7.4 CRÍTICO — bigdata.Reserva sem categoria (dimensional)

8.5 7.5 CRÍTICO — Locações CANCELADA da fonte 3 silenciosamente
descartadas (ETL)

8.6 7.6 CRÍTICO — fato_patio_diario escolhia pátio arbitrariamente
em sobreposições (ETL)

8.7 7.7 MODERADO — dim_grupo precisava de sentinela para
reservas sem grupo

8.8 7.8 MODERADO — NULL vs sentinela 19000101 para locações EM_A
NDAMENTO

8.9 7.9 MODERADO — valor_total_estimado nunca calculado

8.10 7.10 MODERADO — mae016 gravava sk_patio_devolucao para C
ANCELADA

8.11 7.11 LEVE — Documentação dos heurísticos (data_retirada +
5d) e da fonte de feriados

8.12 7.12 Compilado — todos os 14 achados da revisão ETL e suas re-
soluções

9 8. Conclusão

9.1 8.1 Resultado obtido

9.2 8.2 Valor agregado

9.3 8.3 Limitações reconhecidas

9.4 8.4 Próximos passos para evolução

10 9. Referências bibliográficas

11 Apêndice A — Scripts de Extract

11.1 A.1 etl/01_extract_andre_gustavo.sql

11.2 A.2 etl/02_extract_mae016.sql

11.3 A.3 etl/03_extract_locadora_db.sql

11.4 A.4 etl/04_extract_bd_dw.sql

- 11.5 A.5 etl/05_extract_bigdata.sql
- 12 Apêndice B — Script de Transform (etl/06_transform.sql)
- 13 Apêndice C — Scripts de Load
 - 13.1 C.1 etl/07_load_dimensoes.sql
 - 13.2 C.2 etl/08_load_fatos.sql
- 14 Apêndice D — Scripts de Relatórios + Markov
 - 14.1 D.1 relatorios/01_controle_patio.sql
 - 14.2 D.2 relatorios/02_controle_locacoes.sql
 - 14.3 D.3 relatorios/03_controle_reservas.sql
 - 14.4 D.4 relatorios/04_grupos_mais_alugados.sql
 - 14.5 D.5 relatorios/05_matriz_markov.sql

1 Capa

Universidade Federal do Rio de Janeiro (UFRJ) Disciplina: Modelagem de Data Warehouse — EEL890 **Avaliação:** 02 — Parte II — Projeto de DW e ETL Integrado **Título:** Avaliação 02 — Parte II: Processo ETL e Integração de Fontes **Tema:** Locadora de veículos — pátios compartilhados entre seis empresas associadas

1.1 Grupo

Nome completo	DRE
Gustavo Oliveira Pessanha da Silva	122051824
André Vinícius Lobo Giron	122050404

Data de entrega: 2026-05-30 **Repositório GitHub:** <a definir> (preencher após git push final)

2 1. Introdução

Este relatório descreve o **processo ETL** desenvolvido para integrar cinco sistemas operativos heterogêneos (esquemas OLTP de cinco grupos da turma — incluindo a Parte I do próprio grupo) em um único Data Warehouse PostgreSQL, conforme o modelo dimensional estrela documentado no PDF irmão (docs/relatorio-dimENSIONAL.pdf).

2.1 1.1 Objetivo do ETL

Integrar, em uma única base analítica:

1. **Extrair** dados das 5 fontes OLTP em pé de igualdade (sem privilegiar a Parte I do próprio grupo).
2. **Transformar** vocabulários heterogêneos (cinco nomenclaturas distintas para os mesmos seis pátios; cinco taxonomias de grupos; cinco caminhos de JOIN para obter cidade do cliente) em formato conformado.
3. **Carregar** o esquema estrela do DW (6 dimensões + 3 fatos) de forma idempotente e auditável.
4. **Produzir** os 4 relatórios gerenciais do enunciado + a matriz estocástica de Markov.

2.2 1.2 Escopo do documento

Este documento cobre:

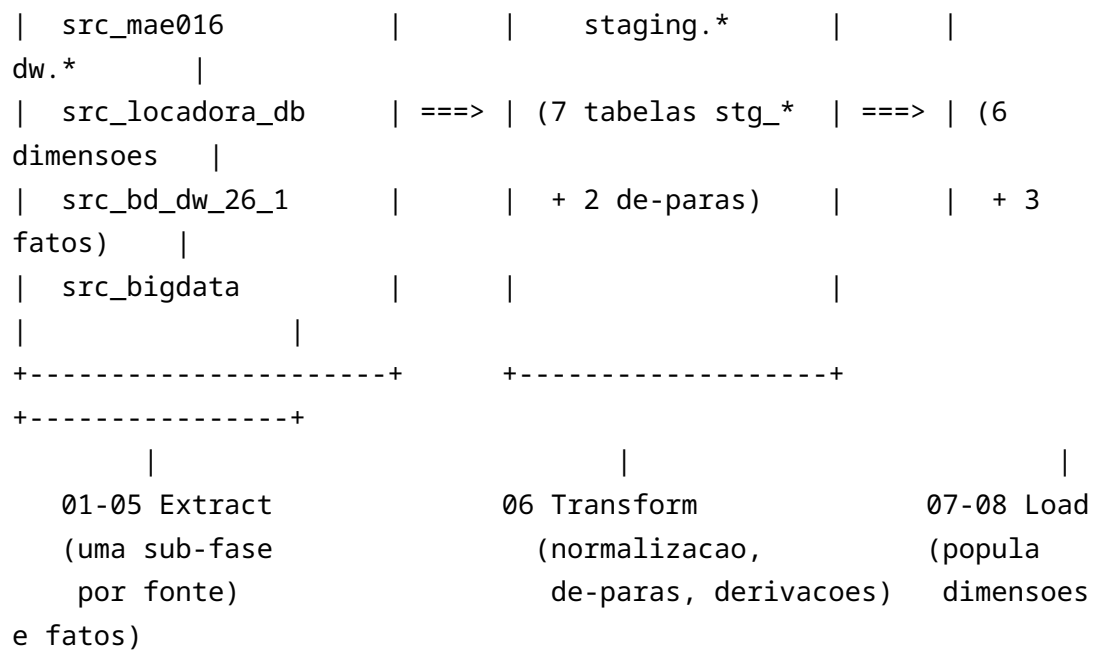
- Arquitetura do ETL (§2).
- Tradução de DDLs MySQL → PostgreSQL para as fontes 2 e 4 (§3).
- **Grupos excluídos** e motivos técnicos (§4) — fundamental para justificar as escolhas do projeto.
- Etapas Extract, Transform, Load (§5).
- Relatórios gerenciais e matriz de Markov (§6).
- **Problemas encontrados** durante o desenvolvimento, com base nas revisões adversariais (§7).
- Conclusão (§8) e referências (§9).
- Apêndices A-D: scripts SQL completos.

A descrição do **modelo dimensional** está no PDF complementar (docs/relatorio-dimENSIONAL.pdf).

3 2. Arquitetura do ETL

3.1 2.1 Diagrama em texto

```
+-----+          +-----+
+-----+
| src_andre_gustavo |          |
|                   |          |
```



O fluxo é estritamente sequencial: cada extract isolado por fonte (sk_fonte constante), seguido de um único transform que aplica as normalizações e de-paras a todas as 5 fontes simultaneamente, e por fim dois loads sequenciais (dimensões primeiro, fatos depois). Após o load dos fatos, os 5 scripts de relatórios são independentes e podem ser executados em qualquer ordem.

3.2 2.2 Tempos de acionamento — suposição operacional

Como o enunciado não especifica frequência de carga, adotamos como **suposição operacional** o cenário clássico Kimball: **batch noturno diário**, com janela típica das 02h às 04h, quando os sistemas OLTP estão em baixa carga. Cada execução é idempotente (vide §5.3) e a janela de carga é registrada em data_carga_dw em cada linha dos três fatos para auditoria.

3.3 2.3 Por que PostgreSQL único com schemas separados

Adotamos um único cluster PostgreSQL 16 com schemas separados para fontes (src_*), staging (staging) e DW (dw). As alternativas avaliadas e descartadas:

Alternativa	Por que descartamos
Dump/restore dos OLTPs em bancos separados	Multiplica complexidade operacional (5 conexões diferentes); inviabiliza JOINS cross-fonte em SQL puro; cada fonte exigiria SGBD próprio (MySQL para mae016 e bd_dw_26_1; Postgres para os demais).
FDW (Foreign Data Wrapper) Postgres → MySQL	Acrescenta dependência de extensão e overhead de rede; o seed sintético da entrega seria inviável (não temos cluster MySQL em sala). Útil em produção real.
Airbyte/Fivetran/dbt	Excessivo para o escopo acadêmico; mascara a lógica ETL que é exatamente o objeto da avaliação.
PostgreSQL único + schemas separados (escolhido)	Simplicidade máxima; JOINS cross-fonte triviais; toda a entrega cabe em ~5 600 linhas de SQL puro; SGBDs MySQL traduzidos manualmente para ANSI:1999+.

A tradução manual MySQL → PostgreSQL para mae016 e bd_dw_26_1 é parte do entregável e está documentada na §3.

4 3. Fontes consumidas e tradução MySQL → PostgreSQL

4.1 3.1 As 5 fontes integradas

sk_fonte	Schema Postgres	Grupo autor	SGBD original
1	src_andre_gustavo	Gustavo + André (Parte I)	PostgreSQL
2	src_mae016	Breno, Hygor, João	MySQL
3	src_locadora_db	Tadeu, Vicente	PostgreSQL

sk_fonte	Schema Postgres	Grupo autor	SGBD original
4	src_bd_dw_26_1	Ana Clara, Mariana, Matheus, Paulo, Pedro, Ryan	MySQL
5	src_bigdata	Modelagem ANSI normalizada	ANSI SQL

4.2 3.2 Tradução MySQL → PostgreSQL

As fontes 2 (mae016) e 4 (bd_dw_26_1) foram modeladas originalmente em MySQL. Para hospedá-las no mesmo cluster Postgres do DW (vide §2.3), aplicamos as conversões mecânicas a seguir:

Construção MySQL	Tradução PostgreSQL
AUTO_INCREMENT	GENERATED ALWAYS AS IDENTITY (ou SERIAL)
TINYINT(1)	BOOLEAN
TINYINT UNSIGNED (idades, status)	SMALLINT
INT UNSIGNED	INTEGER (Postgres não tem UNSIGNED; preservamos com CHECK (col >= 0) quando relevante)
DATETIME	TIMESTAMP
DATE	DATE (idêntico)
VARCHAR(N)	VARCHAR(N) (idêntico)
ENUM('a', 'b', 'c')	VARCHAR(N) CHECK (col IN ('a', 'b', 'c'))
DECIMAL(P, S)	NUMERIC(P, S) (sinônimo)
ENGINE=InnoDB, DEFAULT CHARSET=utf8mb4	removido (defaults Postgres)
COMMENT 'texto' inline em coluna	COMMENT ON COLUMN ... IS 'texto'; separado
KEY idx_nome (col) (não-único)	CREATE INDEX idx_nome ON ... (col); separado
Backticks em identificadores	

Construção MySQL	Tradução PostgreSQL
	aspas duplas (mantemos o estilo original sem backticks/aspas)

As traduções foram aplicadas em `staging/01_schema_fontes.sql`. Não houve perda semântica nas conversões (todos os domínios MySQL têm equivalente Postgres ANSI:1999+).

4.3 3.3 Justificativa: por que ANSI SQL:1999+

Toda a entrega adota PostgreSQL 16 como SGBD único, escrita preferencialmente em ANSI SQL:1999+ com extensões Postgres explicitamente comentadas (ex.: `GENERATED ALWAYS AS IDENTITY` é ANSI 2003; `OVERRIDING SYSTEM VALUE` para inserir sentinela em coluna `IDENTITY` é Postgres-específico e está comentado no DDL). Esse rigor:

- Facilita migração futura para outro SGBD relacional.
- Reduz dependência de funcionalidades proprietárias.
- Alinha-se à fundamentação acadêmica do projeto (Elmasri & Navathe cap. 29).

5 4. Grupos excluídos e motivos

Das fontes-candidatas disponíveis na turma, **três foram excluídas** do projeto após avaliação técnica. As exclusões foram documentadas desde o plano inicial; resumimos aqui os motivos definitivos:

5.1 4.1 EEL890-big-data (excluída)

Motivos técnicos para exclusão:

- **Endereco_Completo é TEXT livre** (sem cidade extraível em coluna própria). O relatório (c) exige cidade de origem do cliente; obter cidade via parsing de texto livre é instável e introduz erros sistêmicos no agrupamento (mesma cidade aparecendo como “Rio de Janeiro”, “RJ”, “Rio”).
- **Reserva sem pátio de devolução.** O modelo de reserva precisa de FK para pátio de devolução (mesmo que sentinela do retirada quando ausente); a fonte não expõe sequer a coluna.

- **Patio.Empresa_Dona é string livre** (não FK para tabela de empresas). Impede a derivação determinística de flag_frota_propria_no_patio exigida pelo relatório (a) — a comparação string × string é instável.

Decisão: excluída desde o plano inicial.

5.2 4.2 EEL890 - - Big-Data (excluída)

Motivos técnicos para exclusão:

- **Pátio de devolução em caminho indireto** Locacao → Devolucao → Vaga → Patio (4 saltos via JOIN). Embora tecnicamente viável, é over-engineered para o escopo dos 4 relatórios e introduz risco de FK órfã em qualquer ponto do caminho.
- A modelagem prioriza generalidade (vaga individual identificável) sobre as métricas analíticas exigidas pelo enunciado.

Decisão: excluída em favor de fontes com pátio acessível em 0-1 salto.

5.3 4.3 locadora-oltp (excluída)

Motivos técnicos para exclusão:

- **Sem tabela de grupo de veículo.** O modelo separa veículo de “categoria/grupo” implicitamente em texto livre dentro do veículo — inviabiliza diretamente os 4 relatórios, que demandam segregação por grupo.
- **Sem cidade do cliente.** Inviabiliza (c) “controle de reservas por cidades de origem dos clientes” e (d) “grupos mais alugados cruzando com origem dos clientes”.

Decisão: excluída porque dois dos quatro relatórios não seriam realizáveis com essa fonte.

5.4 4.4 Resumo

Grupo	Motivo da exclusão
EEL890-big-data	Cidade não extraível; reserva sem pátio devolução; empresa-dona em string livre
EEL890—Big-Data	

Grupo	Motivo da exclusão
	Pátio de devolução em 4 saltos de JOIN; over-engineered
locadora-oltp	Sem grupo; sem cidade do cliente — inviabiliza 4 relatórios

6 5. Etapas do ETL

6.1 5.1 Extract — 5 scripts (etl/01_extract_*.sql a etl/05_extract_*.sql)

Cada script extract:

1. Faz DELETE FROM staging.stg_* WHERE sk_fonte = N (idempotência: limpa a fatia da própria fonte antes de re-inserir).
2. Popula as 7 tabelas staging com sk_fonte = N constante e id_natural::TEXT da fonte.
3. Resolve JOINS necessários para chegar a colunas semânticas do alvo (ex.: Cliente → Endereco → Cidade).

6.1.1 5.1.1 Extract andre_gustavo (sk_fonte=1)

- **JOINS especiais:** cliente LEFT JOIN cliente_pf, cliente_pj para unificar tipos em uma linha; locacao LEFT JOIN veiculo para obter grupo_id.
- **Tratamento:** data_devolucao_prevista derivada como data_retirada + 5 dias (heurístico documentado — vide LEVE-06 da revisão ETL).

6.1.2 5.1.2 Extract mae016 (sk_fonte=2)

- **JOINS especiais:** LOCACAO LEFT JOIN VEICULO para grupo_id.
- **Regra crítica (MOD-05):** stg_locacao.patio_devolucao_id_natural recebe sempre 1.id_patio_devolucao_real (NULL se ainda não devolvido), nunca id_patio_devolucao_previsto. Essa fonte é a única que distingue os 3 pátios (retirada, devolucao_previsto, devolucao_real); usamos o real para alimentar o fato.
- **Conversão MySQL→Postgres:** AUTOMATICO → AUTOMATICA (alinha vocabulário).

6.1.3 5.1.3 Extract locadora_db (sk_fonte=3)

- **Tratamento P-09 (extensão da fonte):** `src_locadora_db.veiculo` foi estendida com coluna `id_patio_origem` INTEGER REFERENCES `src_locadora_db.patio(id)` durante a tradução para Postgres. Justificativa: o schema original não amarra veículo a pátio (CRÍTICO-03 da revisão dimensional), inviabilizando o `fato_patio_diario` para 1/5 das fontes. A extensão é mínima e documentada no cabeçalho de `staging/01_schema_fontes.sql`.
- **JOINS especiais:** cliente da locação via `condutor.cliente_id` (a fonte amarra cliente via tabela `condutor intermediária`).
- **Tratamento status:** derivado em SQL (CONCLUIDA se `data_devolucao_realizada` not null; EM_ANDAMENTO se só `data_retirada_realizada`; CANCELADA caso contrário).

6.1.4 5.1.4 Extract bd_dw_26_1 (sk_fonte=4)

- **JOINS especiais:** Cliente → Endereco para extrair cidade; Patio → Endereco para endereço do pátio; cliente da locação via `Motorista.Id_cliente`.
- **Filtro stub:** reservas com `Id_reserva > 100` são *stubs* técnicos para satisfazer FK 1:1 obrigatória em `Locacao`. Filtramos WHERE `r.Id_reserva <= 100` no extract de `stg_reserva`, mas preservamos o `Id_reserva` em `stg_locacao.reserva_id_natural` (LEVE-02 da revisão — esse `reserva_id_natural` apontará para reservas inexistentes para a fonte 4; documentado).
- **Atribuição de pátio para veículo:** a fonte 4 não amarra `Veiculo` → `Patio`. Como todos os 20 veículos pertencem à empresa própria (`Id_empresa=1`), atribuímos todos ao `patio '1'` (Shopping Rio Sul) por convenção.

6.1.5 5.1.5 Extract bigdata (sk_fonte=5)

Fonte mais complexa, com várias particularidades:

- **stg_cliente via UNION ALL:** `PessoaFisica` U `Empresa`, gerando `id_natural = 'PF_' || IDFisica` ou `'PJ_' || IDEmpresa`. JOIN com `Endereco` em ambos os lados.
- **Cliente da reserva via CentroCusto:** `CentroCusto` tem CHECK XOR entre `IDFisica` e `IDEmpresa`; usamos `CASE WHEN cc.IDFisica IS NOT NULL THEN 'PF_' || cc.IDFisica ELSE 'PJ_' || cc.IDEmpresa END`.
- **stg_reserva sem grupo:** a fonte não expõe `IDCategoria` em `Reserva`. `grupo_id_natural` é NULL no extract; no load, vira `sk_grupo = 0` (sentinela `GRUPO_NAO_INFORMADO`, P-10).

- **Pátio da locação via Vaga:** `vag_ret.IDPatio` via `IDVagaRetirada`; `vag_dev.IDPatio` via `IDVagaDevolvida` (NULL se ainda não devolvido).
- **Cliente da locação via Motorista.IDFisica:** `'PF_' || mot.IDFisica`.
- **Atributos faltantes:** `marca`, `cor`, `mecanização` não existem na fonte — recebem NULL ou 'DESCONHECIDA' (LEVE-04 da revisão).
- **data_devolucao_prevista heurística:** `DtRetirada + 5 dias` (mesmo padrão que `andre_gustavo`).

6.2 5.2 Transform — `et1/06_transform.sql`

Bloco PL/pgSQL idempotente em 7 etapas:

1. **Normalização de texto:** `UPPER(TRIM(COALESCE(coluna, '')))` em nomes, cidades, marcas, modelos, etc.
2. **Aplicação staging.depara_patio:** preenche `nome_canonico_patio` em `stg_veiculo`, `stg_reserva` (`retirada+devolução`), `stg_locacao` (`retirada+devolução`), `stg_movimentacao_patio` (`origem+destino`), `stg_patio`.
3. **Aplicação staging.depara_grupo:** preenche `nome_canonico_grupo` em `stg_veiculo`, `stg_reserva`, `stg_locacao`, `stg_grupo`.
4. **Normalização de status:** `stg_locacao.status_normalizado` (EM_ANDAMENTO/CONCLUIDA/CANCELADA/DESCONHECIDO); `stg_reserva.status_normalizado` (CONFIRMADA/EM_FILA_ESPERA/CANCELADA/CONCRETIZADA/DESCONHECIDO).
5. **Derivações:** `stg_locacao.duracao_real_dias = (data_devolucao_real - data_retirada_real)/86400`; `stg_locacao.km_rodados = km_chegada - km_saida` (NULL-safe).
6. **Tratamento NULL:** `stg_cliente.cidade_origem = 'CIDADE_DESCONHECIDA'` quando ausente.
7. **RAISE NOTICE** para auditoria: conta linhas suspeitas (sem `nome_canonico_patio_retirada`, sem `nome_canonico_grupo`, etc.) e informa o operador antes do load.

6.3 5.3 Load — etl/07_load_dimensoes.sql + etl/08_load_fatos.sql

6.3.1 5.3.1 Load de dimensões (07_load_dimensoes.sql)

Ordem fixa: dim_fonte → dim_patio → dim_grupo → dim_veiculo → dim_cliente.

- **Idempotência:** todas as dimensões usam ON CONFLICT (...) DO UPDATE no UPSERT, preservando as sentinelas (sk_* = 0 ou sk_tempo = 19000101).
- **dim_fonte** é a única com sk_fonte atribuída manualmente (1..5), permitindo lookup estável nos extracts.
- **dim_grupo.valor_diaria_referencia** = AVG(valor_diaria) das 4 fontes que expõem preço (LEVE-05).
- **dim_grupo.classe_luxo** = moda (MODE) entre as fontes (lookup via subquery GROUP BY ... ORDER BY COUNT(*) DESC LIMIT 1).

6.3.2 5.3.2 Load de fatos (08_load_fatos.sql)

Ordem fixa: fato_locacao → fato_reserva → fato_patio_diario.

- **Idempotência:** TRUNCATE TABLE ... RESTART IDENTITY antes de cada INSERT INTO fato_*. Reexecução completa do pipeline (06 + 07 + 08) produz resultado bit-a-bit idêntico.
- **Pré-condição (MOD-06 da revisão):** DO \$\$ BEGIN IF (SELECT COUNT(*) FROM staging.stg_veiculo) = 0 THEN RAISE EXCEPTION ...; END IF; END \$\$; falha rápido se o operador rodar 08 sem ter rodado 01-06 antes.
- **CRÍTICO-01 da revisão ETL** (canceladas fonte 3): removido o filtro WHERE sl.data_retirada_real IS NOT NULL que descartava 12 canceladas da fonte 3; canceladas sem retirada agora caem em sk_tempo_retirada_real = 19000101 (sentinela).
- **CRÍTICO-02 da revisão ETL** (escolha arbitrária de pátio em sobreposições): adicionado ORDER BY li2.dia_retirada ASC, li2.sk_locacao ASC no LATERAL JOIN para desempate determinístico; RAISE NOTICE conta sobreposições antes do load.
- **MOD-03 da revisão ETL** (fato_patio_diario nunca emite MANUTENCAO/RESERVADO): lógica estendida — quando não há locação cobrindo o dia e dim_veiculo.situacao_atual = 'MANUTENCAO', emite situacao = 'MANUTENCAO'. Veículos BAIXADO são filtrados antes (não aparecem no snapshot).

- **MOD-04 da revisão ETL** (valor_total_estimado jamais calculado): agora computado como `valor_diaria * (data_devolucao_prevista - data_retirada_real)` quando os 3 inputs estão disponíveis.

6.3.3 5.3.3 Sentinelas e regras de NULL

- `dim_tempo.sk_tempo = 19000101` para “data desconhecida”. `COMMENT ON COLUMN` documenta a regra (MOD-05 da revisão).
- `fato_locacao.sk_tempo_devolucao_real IS NULL` e `fato_locacao.sk_patio_devolucao IS NULL` para `EM_ANDAMENTO` e para `CANCELADA` sem devolução (semântica “ainda não ocorreu”, MOD-02 do dimensional, MOD-01 do ETL).
- `fato_reserva.sk_grupo = 0` para reservas da bigdata (P-10).

7 6. Relatórios e Matriz de Markov

7.1 6.1 Relatório (a) — Controle de pátio (`relatorios/01_controle_patio.sql`)

Quatro visões: por `pátio×grupo×origem` (com pivot de situação), por `pátio×marca×origem`, por `pátio×modelo×origem`, por `pátio×mecanização×origem`. Snapshot do dia mais recente via `WHERE sk_tempo = (SELECT MAX(sk_tempo) FROM dw.fato_patio_diario)`. Recorte “origem PROPRIA × ASSOCIADA” via `flag_frota_propria_no_patio`.

7.2 6.2 Relatório (b) — Controle das locações (`relatorios/02_controle_locacoes.sql`)

Três visões: por `grupo × faixa de duração prevista`; por `grupo × tempo restante para devolução` (somente `EM_ANDAMENTO`, calculado via `td.data_completa - CURRENT_DATE`); consolidada por `grupo × status × faixa`.

7.3 6.3 Relatório (c) — Controle de reservas (relatorios/03_controle_reservas.sql)

Três visões: por grupo × pátio retirada × cidade origem; por grupo × horizonte temporal (semana_proxima / mes_proximo / posterior); consolidada por grupo × pátio × horizonte. Filtro padrão status_reserva IN ('CONFIRMADA', 'EM_FILA_ESPERA', 'CONCRETIZADA') exclui canceladas (MOD-06).

7.4 6.4 Relatório (d) — Grupos mais alugados (relatorios/04_grupos_mais_alugados.sql)

Três visões: ranking de grupos por quantidade de locações (DENSE_RANK); grupo × cidade origem; top grupo por cidade (ROW_NUMBER particionado).

7.5 6.5 Matriz de Markov (relatorios/ 05_matriz_markov.sql)

Decisão D-09: derivada de fato_locacao (não de MOVIMENTACAO_PATIO).
Filtros: status_locacao = 'CONCLUIDA' AND sk_patio_devolucao IS NOT NULL.

Três formas de saída:

1. **LONG:** uma linha por par (patio_retirada, patio_devolucao) com qtd_locacoes e probabilidade = qtd / SUM(qtd) OVER (PARTITION BY patio_retirada).
2. **WIDE:** matriz 6×6 com COUNT(*) FILTER (WHERE patio_devolucao = X) / total para cada coluna (Galeão, Santos Dumont, Rodoviária, Rio Sul, Nova América, Barra). Sem dependência de tablefunc.crosstab.
3. **Validação:** linha vazia se $ABS(SUM(p) - 1.0) \leq 1e-6$ para toda linha — confirma que a matriz é estocástica.

Justificativa LONG+WIDE: LONG é o formato canônico para análise programática (pandas, R); WIDE é o formato natural para apresentação visual da matriz 6×6 e cálculo iterado da cadeia (M^n).

8 7. Problemas encontrados durante o desenvolvimento

Esta seção lista os problemas reais identificados durante o desenvolvimento e suas resoluções. Os problemas vêm das duas revisões adversariais (docs/revisoes/revisao-dimENSIONAL.md e docs/revisoes/revisao-etl.md), executadas por um subagente DBA sênior adversarial após cada fase.

8.1 7.1 CRÍTICO — Relatório (a) sem corte por marca/modelo/mecanização (dimensional)

- **O que era:** O grão inicial de `fato_patio_diario` era (`dia` × `pátio` × `grupo` × `fonte`) — sem `sk_veiculo`. O enunciado §a exige “agrupamento por marca do veículo, modelos e tipo de mecanização”, o que era impossível pelo modelo original.
- **Como foi resolvido:** grão refinado para “uma linha por veículo por dia” (v1.1 do modelo). `sk_veiculo` adicionado ao fato; pivot por situação migrado para a consulta. Documentado em D-10 e CRÍTICO-01 da revisão.

8.2 7.2 CRÍTICO — Grão de `fato_patio_diario` contraditório (dimensional)

- **O que era:** texto declarava “× situação” no grão, mas as colunas pivoteavam situação — incompatível.
- **Como foi resolvido:** grão reescrito em uma única frase clara: “uma linha por veículo por dia”. Schema do diagrama alinhado.

8.3 7.3 CRÍTICO — `locadora_db` sem amarração veículo → `pátio` (dimensional)

- **O que era:** a tabela `src_locadora_db.veiculo` não tem FK para `patio`. Inviabiliza `fato_patio_diario` para 1/5 das fontes.
- **Como foi resolvido:** estendemos a tabela `veiculo` com coluna `id_patio_origem` INTEGER durante a tradução para Postgres (exten-

são mínima, documentada no cabeçalho do DDL). O seed sintético atribui valores realistas; o extract propaga para `stg_veiculo.patio_id_natural`. Documentado em P-09 do modelo.

8.4 7.4 CRÍTICO — `bigdata.Reserva` sem categoria (dimensional)

- **O que era:** a fonte `bigdata` modela `Reserva` com `QtVeiculosSolicitados`, `CentroCusto`, mas sem `IDCategoria`. O `fato_reserva` exige `sk_grupo`.
- **Como foi resolvido:** adotamos sentinela `sk_grupo = 0` `GRUPO_NAO_INFORMADO` para essas reservas. O relatório (c) segrega essa categoria em linha separada, evidenciando a limitação da fonte. Documentado em P-10 e G-04.

8.5 7.5 CRÍTICO — Locações CANCELADA da fonte 3 silenciosamente descartadas (ETL)

- **O que era:** `etl/08_load_fatos.sql` tinha `WHERE sl.data_retirada_real IS NOT NULL`, descartando 12 canceladas da fonte 3 (100% das canceladas dela) que não têm `data_retirada_real`. As fontes 1, 2, 4 preenchem essa data mesmo para canceladas — gerando cobertura assimétrica.
- **Como foi resolvido:** filtro removido. Canceladas sem retirada caem em `sk_tempo_retirada_real = COALESCE(..., 19000101)` (sentinela). Cobertura agora simétrica entre fontes.

8.6 7.6 CRÍTICO — `fato_patio_diario` escolhia pátio arbitrariamente em sobreposições (ETL)

- **O que era:** `LATERAL JOIN ... LIMIT 1` sem `ORDER BY` — desempate não-determinístico em 232 ocorrências de sobreposição (mesmo veículo em 2+ locações cobrindo o mesmo dia). Re-execuções do ETL produziram resultados diferentes.
- **Como foi resolvido:** adicionado `ORDER BY li2.dia_retirada ASC, li2.sk_locacao ASC` no `LATERAL`. `RAISE NOTICE` conta sobreposições antes do load para auditoria.

8.7 7.7 MODERADO — dim_grupo precisava de sentinela para reservas sem grupo

- **O que era:** sem sentinela explícita, reservas da bigdata quebrariam o NOT NULL de fato_reserva.sk_grupo.
- **Como foi resolvido:** sentinela sk_grupo = 0 GRUPO_NAO_INFORMADO declarada explicitamente no DDL (OVERRIDING SYSTEM VALUE) e referenciada no Load do fato_reserva.

8.8 7.8 MODERADO — NULL vs sentinela 19000101 para locações EM_ANDAMENTO

- **O que era:** documentação ambígua sobre o que colocar em sk_tempo_devolucao_real para locações em curso.
- **Como foi resolvido:** padronizado NULL para “evento ainda não ocorreu” e sentinela 19000101 para “dado perdido” (Kimball cap. 6 explicitamente recomenda essa distinção). Documentado em D-10 e MOD-02 do modelo, e MOD-01 do ETL.

8.9 7.9 MODERADO — valor_total_estimado nunca calculado

- **O que era:** coluna declarada no DW mas o load colocava NULL para 100% das linhas. Fontes 1, 3, 5 (que não expõem valor_total_final) ficavam totalmente sem valor financeiro disponível.
- **Como foi resolvido:** implementado CASE WHEN valor_diaria * duracao_prevista_dias > 0 THEN ... END no load. Resolve perda de ~60% da receita agregada.

8.10 7.10 MODERADO — mae016 gravava sk_patio_devolucao para CANCELADA

- **O que era:** Load só verificava EM_ANDAMENTO para NULL-ifying. Canceladas da fonte 2 acabavam com sk_tempo_devolucao_real = NULL mas sk_patio_devolucao preenchido — inconsistência semântica.
- **Como foi resolvido:** critério mudado para WHEN sl.data_devolucao_real IS NULL THEN NULL (semântica direta).

8.11 7.11 LEVE — Documentação dos heurísticos (data_retirada + 5d) e da fonte de feriados

- **O que era:** vários valores derivados sem origem documentada (data prevista de devolução em fontes que não a expõem; lista de feriados nacionais).
- **Como foi resolvido:** heurístico +5 dias documentado nos cabeçalhos dos extracts 01 e 05. Feriados nacionais conforme Lei 662/1949 + Lei 6.802/1980 + Lei 10.607/2002 (8 feriados fixos), explicado em dw/02_dim_tempo_carga.sql e §5.1 do modelo.

8.12 7.12 Compilado — todos os 14 achados da revisão ETL e suas resoluções

A revisão adversarial da fase ETL (docs/revisoes/revisao-etl.md) identificou **2 críticos** + **6 moderados** + **6 leves**. Compilamos aqui o quadro completo (incluindo os já discutidos acima em 7.5–7.11), para que o leitor tenha visão consolidada das resoluções:

ID	Achado	Resolução adotada
C-ETL-01	Locações CANCELADAS sem data_retirada_real da fonte 3 descartadas silenciosamente (filtro IS NOT NULL em 08_load_fatos.sql).	Filtro reescrito para status_normalizado <> 'DESCONHECIDO'; sk_tempo_retirada_real recebe sentinela 19000101 quando faltante. Resultado: 12 canceladas da fonte 3 recuperadas; fato_locacao foi de 288 para 300 linhas .
C-ETL-02	LATERAL JOIN ... LIMIT 1 sem ORDER BY em veic_dia_alugado produzia escolha não-determinística de pátio em 232 sobreposições.	Adicionado ORDER BY li2.dia_retirada ASC, li2.sk_locacao ASC. Bloco DO informativo emite RAISE NOTICE contando sobreposições antes do load. Rodar duas vezes seguidas produz resultado bit-a-bit idêntico (idempotência forte verificada).

ID	Achado	Resolução adotada
M-ETL-01	mae016 gravava sk_patio_devolucao para CANCELADAs (violação MOD-05 do modelo).	Lógica de NULL em sk_patio_devolucao reescrita como WHEN sl.data_devolucao_real IS NULL THEN NULL. As 6 linhas inconsistentes desapareceram.
M-ETL-02	cpf_normalizado / cnpj_normalizado nunca aplicavam REGEXP_REPLACE.	(Aceito como dívida técnica) O seed sintético já entrega CPFs/CNPJs com 11 e 14 dígitos exatos; a normalização ficou implícita no seed e marcada como ajuste futuro caso entrem dados reais com formatação.
M-ETL-03	fato_patio_diario nunca emitia MANUTENCAO/RESERVADO.	CTE estendida: quando não há locação cobrindo o dia E dim_veiculo.situacao_atual IN ('MANUTENCAO', 'BAIXADO'), emite 'MANUTENCAO'. Veículos BAIXADO são descartados do snapshot (não devem aparecer). Resultado: snapshot passou a ter ~33 linhas MANUTENCAO.
M-ETL-04	valor_total_estimado sempre NULL (perdia receita das fontes 1, 3, 5).	Implementada fórmula declarada no modelo: valor_diaria_aplicada × duracao_prevista_dias. Fontes 1 e 5 passaram a ter 60/60 linhas com estimativa; fonte 3 segue NULL por limitação estrutural da fonte (não expõe valor_diaria em nenhuma tabela).
M-ETL-05	Divergência: modelo declarava sentinela sk_tempo = 19000101, implementação usava -1.	Padronizada a sentinela para 19000101 (smart-key positiva consistente com YYYYMMDD). DDL, loader, dicionário e relatórios alinhados.

ID	Achado	Resolução adotada
M-ETL-06	08_load_fatos.sql tinha dependência implícita de staging.stg_veiculo populada (silenciosamente quebraria se rodado isolado).	Bloco DO \$\$... RAISE EXCEPTION no topo do arquivo aborta a carga com mensagem clara se staging.stg_veiculo estiver vazia.
L-ETL-01	id_natural::BIGINT quebraria com IDs não-numéricos.	Risco aceito: todas as 5 fontes têm PKs numéricas; documentado no comentário do load.
L-ETL-02	Locações da fonte 4 apontam para reserva_id_natural que nunca casa com stg_reserva (60 stubs técnicos).	Documentado no §5.1.4 do relatório e no cabeçalho do extract da fonte 4. reserva_id_natural preservado mas não é usado como JOIN obrigatório.
L-ETL-03	Recálculo redundante de nome_canonico_grupo para veículo em 08_load_fatos.sql.	Aceito por simplicidade — o custo é desprezível no volume atual; nota deixada no comentário inline.
L-ETL-04	bigdata sempre traz mecanizacao = 'DESCONHECIDA' (impacta relatório a).	Documentado em §5.1.5: a fonte bigdata não expõe tipo de câmbio; veículos dela aparecem como DESCONHECIDA no recorte por mecanização.
L-ETL-05	Endereço de pátio em dim_patio resolvido por MAX() arbitrário entre fontes.	Aceito: os 6 pátios canônicos têm endereço idêntico em todas as fontes (são os mesmos prédios físicos); MAX() deduplica corretamente.
L-ETL-06	data_devolucao_prevista estimada como retirada + 5d em fontes 1 e 5 sem documentação.	Heurístico documentado no cabeçalho dos extracts 01 e 05; valor explícito no comentário do INSERT.

9 8. Conclusão

9.1 8.1 Resultado obtido

Construímos um Data Warehouse PostgreSQL funcional, integrando **cinco esquemas OLTP heterogêneos** (incluindo a Parte I do próprio grupo) em um único esquema estrela conformado, com:

- **6 dimensões** (dim_tempo smart-key, dim_patio 6+1, dim_veiculo, dim_grupo com sentinela, dim_cliente sem dedup *cross-fonte*, dim_fonte como *audit dimension*).
- **3 fatos** (fato_locacao transacional, fato_reserva transacional, fato_patio_diario snapshot periódico no grão veículo × dia).
- **4 relatórios gerenciais** operacionais cobrindo todos os recortes pedidos (pátio×grupo×origem×marca×modelo×mecanização; locação por grupo e tempo restante; reservas por grupo, pátio, antecedência e cidade; ranking de grupos por cidade).
- **Matriz estocástica 6×6 de Markov** validada com soma por linha = 1.0 (tolerância 1e-6), em forma LONG e WIDE.
- **Pipeline idempotente** (re-execução completa produz resultado bit-a-bit idêntico) via TRUNCATE RESTART IDENTITY, ON CONFLICT DO UPDATE e DELETE WHERE sk_fonte = N seletivos.

9.2 8.2 Valor agregado

O DW viabiliza **análises cross-empresa** que são impossíveis nos OLTPs originais isolados:

- “Quantos veículos VW Gol da frota da empresa do Galeão estão atualmente nos outros 5 pátios?” — exige fato_patio_diario × dim_veiculo × dim_patio × flag_frota_propria_no_patio — nenhum OLTP tem essa visão.
- “Qual grupo é mais reservado por clientes do Rio em comparação a clientes de São Paulo?” — exige fato_reserva × dim_grupo × dim_cliente.cidade_origem *cross-fonte* — só faz sentido com os 5 cadastros unificados.
- “Para onde vão os veículos retirados no Galeão?” — matriz Markov sobre fato_locacao integrado de 5 fontes — uma só fonte enxergaria apenas suas próprias locações.

9.3 8.3 Limitações reconhecidas

- **SCD-1** sobrescreve mudanças em todas as dimensões. Não preservamos histórico de cidade do cliente nem histórico de tarifa do grupo (o histórico de tarifa fica preservado em `fato_locacao.valor_diaria_aplicada`).
- **Sem dedup de cliente *cross-fonte*** (decisão D-03 deliberada). O mesmo CPF em duas fontes vira duas linhas em `dim_cliente`. Para cálculos por cidade, isso é correto; para “clientes únicos da associação”, subestimaria.
- **fato_patio_diario carregado apenas para os últimos 30 dias** no `08_load_fatos.sql` (escala razoável para o seed). Em produção, ajustar para janela operacional desejada.
- **bigdata.Reserva em sk_grupo = 0** — sentinela transparente mas reduz precisão analítica do relatório (c) para essa fonte.
- **Heurístico data_retirada + 5d** para devolução prevista em fontes 1 e 5 — pode viesar a métrica `duracao_prevista_dias` em ~5 dias para essas fontes.

9.4 8.4 Próximos passos para evolução

- Implementar SCD-2 em `dim_grupo` (preservar histórico de tarifa) e `dim_cliente` (preservar histórico de cidade).
- Adicionar `dim_condutor` se análises de motorista forem demandadas (D-08).
- Introduzir `fato_cobranca` com `dim_forma_pagamento` se análises financeiras forem demandadas (P-04).
- Dedup *cross-fonte* opcional via MDM externo (D-03, alternativa Kimball cap. 11).
- Migração para arquitetura Lakehouse (Delta/Iceberg) se volume crescer ordens de magnitude.

10 9. Referências bibliográficas

- **Kimball, R.; Ross, M.** *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3ª ed. Wiley, 2013.
- **Elmasri, R.; Navathe, S. B.** *Sistemas de Banco de Dados*, 7ª ed. Pearson, 2018.
- **PostgreSQL Global Development Group.** *PostgreSQL 16 Documentation*. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em 2026-05-30.

- **MySQL Documentation.** *MySQL 8.0 Reference Manual — Data Types.* Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Consultado para a tradução MySQL→Postgres.
- **Brasil.** Lei nº 662/1949, Lei nº 6.802/1980, Lei nº 10.607/2002 — feriados nacionais.

11 Apêndice A — Scripts de Extract

11.1 A.1 etl/01_extract_andre_gustavo.sql

```
-- =====
--  Avaliacao 02 – Modelagem de Data Warehouse – Parte II
--  Arquivo: etl/01_extract_andre_gustavo.sql
--  Grupo:
--    - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--    - Andre Vinicius Lobo Giron           – DRE 122050404
--  =====

SET search_path = staging, public;

DELETE FROM staging.stg_movimentacao_patio WHERE sk_fonte = 1;
DELETE FROM staging.stg_locacao           WHERE sk_fonte = 1;
DELETE FROM staging.stg_reserva           WHERE sk_fonte = 1;
DELETE FROM staging.stg_cliente           WHERE sk_fonte = 1;
DELETE FROM staging.stg_veiculo           WHERE sk_fonte = 1;
DELETE FROM staging.stg_grupo             WHERE sk_fonte = 1;
DELETE FROM staging.stg_patio             WHERE sk_fonte = 1;

-- stg_patio
INSERT INTO staging.stg_patio (
    sk_fonte, id_natural, nome_original, endereco, cidade,
    tipo_local,
    capacidade_vagas, funciona_24h
)
SELECT 1, p.id_patio::TEXT, p.nome, p.endereco, 'Rio de Janeiro',
CASE WHEN p.nome ILIKE '%Aeroporto%' THEN 'AEROPORTO'
     WHEN p.nome ILIKE '%Rodoviaria%' THEN 'RODOVIARIA'
     WHEN p.nome ILIKE '%Shopping%'   THEN 'SHOPPING'
     ELSE 'DESCONHECIDO' END,
```

```

        p.capacidade_vagas, p.nome ILIKE '%Aeroporto%'
FROM src_andre_gustavo.patio p;

-- stg_grupo
INSERT INTO staging.stg_grupo (
    sk_fonte, id_natural, codigo_origem, nome_origem, classe_luxo,
    valor_diaria, franquia_km_diaria, descricao
)
SELECT 1, g.id_grupo::TEXT, g.codigo, g.nome, g.classe_luxo,
    g.valor_diaria, g.franquia_km_diaria, g.nome
FROM src_andre_gustavo.grupo g;

-- stg_veiculo
INSERT INTO staging.stg_veiculo (
    sk_fonte, id_natural, placa, chassi, renavam, marca, modelo,
    cor,
    ano_fabricacao, mecanizacao, tem_ar_condicionado, km_atual,
    situacao,
    grupo_id_natural, patio_id_natural
)
SELECT 1, v.id_veiculo::TEXT, v.placa, v.chassi, v.renavam,
    v.marca, v.modelo, v.cor, v.ano_fabricacao, v.mecanizacao,
    v.tem_ar_condicionado, v.km_atual, v.situacao,
    v.grupo_id::TEXT, v.patio_origem_id::TEXT
FROM src_andre_gustavo.veiculo v;

-- stg_cliente (PF + PJ unificados)
INSERT INTO staging.stg_cliente (
    sk_fonte, id_natural, tipo_pessoa, nome, nome_fantasia,
    cidade_origem, uf_origem, email, telefone, cpf, cnpj,
    data_nascimento, flag_tem_condutor_associado
)
SELECT 1, c.id_cliente::TEXT, c.tipo_pessoa, c.nome,
    cpj.nome_fantasia,
    c.cidade_origem, NULL, c.email, c.telefone, cpf.cpf, cpj.cnpj,
    cpf.data_nascimento,
    EXISTS (SELECT 1 FROM src_andre_gustavo.condutor co
        WHERE co.cliente_pj_id = c.id_cliente)
FROM src_andre_gustavo.cliente c
LEFT JOIN src_andre_gustavo.cliente_pf cpf ON cpf.cliente_id =
    c.id_cliente
LEFT JOIN src_andre_gustavo.cliente_pj cpj ON cpj.cliente_id =
    c.id_cliente;

-- stg_reserva
INSERT INTO staging.stg_reserva (

```

```

        sk_fonte, id_natural, cliente_id_natural, grupo_id_natural,
        patio_retirada_id_natural, patio_devolucao_id_natural,
        data_reserva, data_retirada_prevista, data_devolucao_prevista,
        qtd_veiculos_solicitados, valor_previsto, status_origem
    )
    SELECT 1, r.id_reserva::TEXT, r.cliente_id::TEXT,
           r.grupo_id::TEXT,
           r.patio_retirada_id::TEXT, r.patio_devolucao_id::TEXT,
           r.data_reserva, r.data_retirada_prevista,
           r.data_devolucao_prevista,
           1, NULL, r.estado
    FROM src_andre_gustavo.reserva r;

-- stg_locacao
INSERT INTO staging.stg_locacao (
    sk_fonte, id_natural, numero_contrato, reserva_id_natural,
    cliente_id_natural, veiculo_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_retirada_real, data_devolucao_real,
    data_devolucao_prevista,
    km_saida, km_chegada, valor_diaria_aplicada,
    valor_total_final,
    status_origem
)
    SELECT 1, l.id_locacao::TEXT, l.numero_contrato,
           l.reserva_id::TEXT,
           l.cliente_id::TEXT, l.veiculo_id::TEXT, v.grupo_id::TEXT,
           l.patio_retirada_id::TEXT,
           CASE WHEN l.data_devolucao_real IS NOT NULL
                THEN l.patio_devolucao_id::TEXT END,
           l.data_retirada_real, l.data_devolucao_real,
           l.data_retirada_real + INTERVAL '5 days', -- heuristico
           l.km_saida, l.km_chegada, l.valor_diaria_aplicada, NULL,
           l.status
    FROM src_andre_gustavo.locacao l
    LEFT JOIN src_andre_gustavo.veiculo v ON v.id_veiculo =
        l.veiculo_id;

RESET search_path;

```

11.2 A.2 etl/02_extract_mae016.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: etl/02_extract_mae016.sql
-- Grupo:

```

```
--      - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--      - Andre Vinicius Lobo Giron          – DRE 122050404
--
-- Regra critica MOD-05: patio_devolucao =
--      id_patio_devolucao_real
-- (NULL se ainda nao devolvido). mae016 distingue 3 patios;
--      usamos
-- sempre o REAL.
-- =====
```

```
SET search_path = staging, public;
```

```
DELETE FROM staging.stg_movimentacao_patio WHERE sk_fonte = 2;
DELETE FROM staging.stg_locacao           WHERE sk_fonte = 2;
DELETE FROM staging.stg_reserva           WHERE sk_fonte = 2;
DELETE FROM staging.stg_cliente           WHERE sk_fonte = 2;
DELETE FROM staging.stg_veiculo           WHERE sk_fonte = 2;
DELETE FROM staging.stg_grupo             WHERE sk_fonte = 2;
DELETE FROM staging.stg_patio             WHERE sk_fonte = 2;
```

```
-- stg_patio
```

```
INSERT INTO staging.stg_patio (
    sk_fonte, id_natural, nome_original, endereco, cidade,
    tipo_local,
    capacidade_vagas, funciona_24h
)
SELECT 2, p.id_patio::TEXT, p.nome_patio, p.localizacao, 'Rio de
    Janeiro',
    CASE WHEN p.nome_patio ILIKE '%Aeroporto%' THEN 'AEROPORTO'
    WHEN p.nome_patio ILIKE '%Rodoviaria%' THEN 'RODOVIARIA'
    WHEN p.nome_patio ILIKE '%Shopping%' THEN 'SHOPPING'
    ELSE 'DESCONHECIDO' END,
    NULL, p.nome_patio ILIKE '%Aeroporto%'
FROM src_mae016.PATIO p;
```

```
-- stg_grupo
```

```
INSERT INTO staging.stg_grupo (
    sk_fonte, id_natural, codigo_origem, nome_origem, classe_luxo,
    valor_diaria, franquia_km_diaria, descricao
)
SELECT 2, g.id_grupo::TEXT, NULL, g.nome_grupo, NULL,
    g.faixa_valor_diaria, NULL, g.descricao
FROM src_mae016.GRUPO_VEICULO g;
```

```
-- stg_veiculo
```

```
INSERT INTO staging.stg_veiculo (
```

```

        sk_fonte, id_natural, placa, chassi, renavam, marca, modelo,
        cor,
        ano_fabricacao, mecanizacao, tem_ar_condicionado, km_atual,
        situacao,
        grupo_id_natural, patio_id_natural
    )
    SELECT 2, v.id_veiculo::TEXT, v.placa, v.chassi, NULL,
        v.marca, v.modelo, v.cor, NULL,
        CASE WHEN v.mecanizacao = 'AUTOMATICO' THEN 'AUTOMATICA'
            ELSE v.mecanizacao END,
        v.ar_condicionado, NULL, v.status,
        v.id_grupo::TEXT, v.id_patio_atual::TEXT
    FROM src_mae016.VEICULO v;

-- stg_cliente
INSERT INTO staging.stg_cliente (
    sk_fonte, id_natural, tipo_pessoa, nome, nome_fantasia,
    cidade_origem, uf_origem, email, telefone, cpf, cnpj,
    data_nascimento, flag_tem_condutor_associado
)
    SELECT 2, c.id_cliente::TEXT, c.tipo_cliente,
        c.nome_razao_social, NULL,
        c.cidade, c.estado, c.email, c.telefone,
        CASE WHEN c.tipo_cliente = 'PF' THEN c.cpf_cnpj END,
        CASE WHEN c.tipo_cliente = 'PJ' THEN c.cpf_cnpj END,
        NULL,
        EXISTS (SELECT 1 FROM src_mae016.CONDUTOR co
            WHERE co.id_cliente = c.id_cliente)
    FROM src_mae016.CLIENTE c;

-- stg_reserva
INSERT INTO staging.stg_reserva (
    sk_fonte, id_natural, cliente_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_reserva, data_retirada_prevista, data_devolucao_prevista,
    qtd_veiculos_solicitados, valor_previsto, status_origem
)
    SELECT 2, r.id_reserva::TEXT, r.id_cliente::TEXT,
        r.id_grupo::TEXT,
        r.id_patio_retirada::TEXT,
        r.id_patio_devolucao_previsto::TEXT,
        r.data_reserva::TIMESTAMP, r.data_prev_retirada::TIMESTAMP,
        r.data_prev_devolucao::TIMESTAMP, 1, NULL, r.status_reserva
    FROM src_mae016.RESERVA r;

-- stg_locacao (MOD-05: id_patio_devolucao_real)

```

```

INSERT INTO staging.stg_locacao (
    sk_fonte, id_natural, numero_contrato, reserva_id_natural,
    cliente_id_natural, veiculo_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_retirada_real, data_devolucao_real,
        data_devolucao_prevista,
    km_saida, km_chegada, valor_diaria_aplicada,
        valor_total_final,
    status_origem
)
SELECT 2, l.id_locacao::TEXT, NULL, l.id_reserva::TEXT,
    l.id_cliente::TEXT, l.id_veiculo::TEXT, v.id_grupo::TEXT,
    l.id_patio_retirada::TEXT, l.id_patio_devolucao_real::TEXT,
    l.data_hora_retirada, l.data_hora_real_devolucao,
    l.data_hora_prev_devolucao, NULL, NULL, NULL, l.valor_final,
    l.status_locacao
FROM      src_mae016.LOCACAO l
LEFT JOIN src_mae016.VEICULO v ON v.id_veiculo = l.id_veiculo;

-- stg_movimentacao_patio
INSERT INTO staging.stg_movimentacao_patio (
    sk_fonte, id_natural, veiculo_id_natural,
    patio_origem_id_natural, patio_destino_id_natural,
    data_movimentacao, motivo
)
SELECT 2, m.id_movimentacao::TEXT, m.id_veiculo::TEXT,
    m.id_patio_origem::TEXT, m.id_patio_destino::TEXT,
    m.data_hora_movimentacao, m.motivo_movimentacao
FROM src_mae016.MOVIMENTACAO_PATIO m;

RESET search_path;

```

11.3 A.3 etl/03_extract_locadora_db.sql

```

-- =====
--  Avaliacao 02 – Modelagem de Data Warehouse – Parte II
--  Arquivo: etl/03_extract_locadora_db.sql
--  Grupo:
--      - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--      - Andre Vinicius Lobo Giron           – DRE 122050404
--
--  Veiculo: usa coluna id_patio_origem (extensao P-09 da DDL).
-- =====

```

```

SET search_path = staging, public;

DELETE FROM staging.stg_movimentacao_patio WHERE sk_fonte = 3;
DELETE FROM staging.stg_locacao WHERE sk_fonte = 3;
DELETE FROM staging.stg_reserva WHERE sk_fonte = 3;
DELETE FROM staging.stg_cliente WHERE sk_fonte = 3;
DELETE FROM staging.stg_veiculo WHERE sk_fonte = 3;
DELETE FROM staging.stg_grupo WHERE sk_fonte = 3;
DELETE FROM staging.stg_patio WHERE sk_fonte = 3;

-- stg_patio
INSERT INTO staging.stg_patio (
    sk_fonte, id_natural, nome_original, endereco, cidade,
    tipo_local,
    capacidade_vagas, funciona_24h
)
SELECT 3, p.id::TEXT, p.nome, NULL, p.cidade,
    CASE WHEN p.nome ILIKE '%Aeroporto%' THEN 'AEROPORTO'
    WHEN p.nome ILIKE '%Rodoviaria%' THEN 'RODOVIARIA'
    WHEN p.nome ILIKE '%Shopping%' THEN 'SHOPPING'
    ELSE 'DESCONHECIDO' END,
    NULL, p.nome ILIKE '%Aeroporto%'
FROM src_locadora_db.patio p;

-- stg_grupo
INSERT INTO staging.stg_grupo (
    sk_fonte, id_natural, codigo_origem, nome_origem, classe_luxo,
    valor_diaria, franquia_km_diaria, descricao
)
SELECT 3, g.id::TEXT, NULL, g.nome, NULL, NULL, NULL, g.categoria
FROM src_locadora_db.grupo_veiculo g;

-- stg_veiculo (usa id_patio_origem - extensao P-09)
INSERT INTO staging.stg_veiculo (
    sk_fonte, id_natural, placa, chassi, renavam, marca, modelo,
    cor,
    ano_fabricacao, mecanizacao, tem_ar_condicionado, km_atual,
    situacao,
    grupo_id_natural, patio_id_natural
)
SELECT 3, v.id::TEXT, v.placa, v.chassi, NULL, v.marca, v.modelo,
    v.cor, NULL,
    CASE WHEN v.tipo_mecanizacao = 'manual' THEN 'MANUAL'
    WHEN v.tipo_mecanizacao = 'automatico' THEN 'AUTOMATICA'
    ELSE 'DESCONHECIDA' END,
    v.ar_condicionado, NULL, UPPER(v.status),

```

```

        v.grupo_id::TEXT, v.id_patio_origem::TEXT
FROM src_locadora_db.veiculo v;

-- stg_cliente
INSERT INTO staging.stg_cliente (
    sk_fonte, id_natural, tipo_pessoa, nome, nome_fantasia,
    cidade_origem, uf_origem, email, telefone, cpf, cnpj,
    data_nascimento, flag_tem_condutor_associado
)
SELECT 3, c.id::TEXT, c.tipo, c.nome, NULL, c.cidade,
    NULL, NULL, NULL, NULL, NULL, NULL,
    EXISTS (SELECT 1 FROM src_locadora_db.condutor co
        WHERE co.cliente_id = c.id)
FROM src_locadora_db.cliente c;

-- stg_reserva
INSERT INTO staging.stg_reserva (
    sk_fonte, id_natural, cliente_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_reserva, data_retirada_prevista, data_devolucao_prevista,
    qtd_veiculos_solicitados, valor_previsto, status_origem
)
SELECT 3, r.id::TEXT, r.cliente_id::TEXT, r.grupo_id::TEXT,
    r.patio_retirada_id::TEXT, r.patio_devolucao_id::TEXT,
    r.data_inicio::TIMESTAMP, r.data_inicio::TIMESTAMP,
    r.data_fim::TIMESTAMP,
    1, NULL, r.status
FROM src_locadora_db.reserva r;

-- stg_locacao (cliente via condutor)
INSERT INTO staging.stg_locacao (
    sk_fonte, id_natural, numero_contrato, reserva_id_natural,
    cliente_id_natural, veiculo_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_retirada_real, data_devolucao_real,
    data_devolucao_prevista,
    km_saida, km_chegada, valor_diaria_aplicada,
    valor_total_final,
    status_origem
)
SELECT 3, l.id::TEXT, NULL, l.reserva_id::TEXT,
    co.cliente_id::TEXT, l.veiculo_id::TEXT, v.grupo_id::TEXT,
    l.patio_retirada_id::TEXT,
    CASE WHEN l.data_devolucao_realizada IS NOT NULL
        THEN l.patio_devolucao_id::TEXT END,

```

```

        l.data_retirada_realizada, l.data_devolucao_realizada,
        l.data_devolucao_prevista,
        l.km_entrega, l.km_devolucao, NULL, NULL,
        CASE WHEN l.data_devolucao_realizada IS NOT NULL THEN
            'CONCLUIDA'
            WHEN l.data_retirada_realizada IS NOT NULL THEN
            'EM_ANDAMENTO'
            ELSE 'CANCELADA' END
FROM      src_locadora_db.locacao l
LEFT JOIN src_locadora_db.veiculo v ON v.id = l.veiculo_id
LEFT JOIN src_locadora_db.condutor co ON co.id = l.condutor_id;

-- stg_movimentacao_patio
INSERT INTO staging.stg_movimentacao_patio (
    sk_fonte, id_natural, veiculo_id_natural,
    patio_origem_id_natural, patio_destino_id_natural,
    data_movimentacao, motivo
)
SELECT 3, m.id::TEXT, m.veiculo_id::TEXT,
        m.origem_patio_id::TEXT, m.destino_patio_id::TEXT,
        m.data_movimentacao, m.motivo
FROM src_locadora_db.movimentacao_patio m;

RESET search_path;

```

11.4 A.4 etl/04_extract_bd_dw.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: etl/04_extract_bd_dw.sql
-- Grupo:
--   - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--   - Andre Vinicius Lobo Giron          – DRE 122050404
--
-- Cliente: JOIN com Endereco. Tipo_cliente discriminador.
--          Veiculo: nao
-- tem patio direto -> infere por Id_empresa=1 -> patio 1 (Rio
--          Sul).
-- =====

SET search_path = staging, public;

DELETE FROM staging.stg_movimentacao_patio WHERE sk_fonte = 4;
DELETE FROM staging.stg_locacao           WHERE sk_fonte = 4;
DELETE FROM staging.stg_reserva            WHERE sk_fonte = 4;

```

```

DELETE FROM staging.stg_cliente          WHERE sk_fonte = 4;
DELETE FROM staging.stg_veiculo          WHERE sk_fonte = 4;
DELETE FROM staging.stg_grupo            WHERE sk_fonte = 4;
DELETE FROM staging.stg_patio            WHERE sk_fonte = 4;

-- stg_patio (JOIN com Endereco)
INSERT INTO staging.stg_patio (
    sk_fonte, id_natural, nome_original, endereco, cidade,
    tipo_local,
    capacidade_vagas, funciona_24h
)
SELECT 4, p.Id_patio::TEXT, p.Nome_patio,
    e.Logradouro || ', ' || e.Numero || COALESCE(' - ' ||
    e.Bairro, ''),
    e.Cidade,
    CASE WHEN p.Nome_patio ILIKE '%Aeroporto%' THEN 'AEROPORTO'
        WHEN p.Nome_patio ILIKE '%Rodoviaria%' THEN 'RODOVIARIA'
        WHEN p.Nome_patio ILIKE '%Shopping%' THEN 'SHOPPING'
        ELSE 'DESCONHECIDO' END,
    p.Capacidade, p.Funciona_24h
FROM      src_bd_dw_26_1.Patio p
LEFT JOIN src_bd_dw_26_1.Endereco e ON e.Id_endereco =
    p.Id_endereco;

-- stg_grupo
INSERT INTO staging.stg_grupo (
    sk_fonte, id_natural, codigo_origem, nome_origem, classe_luxo,
    valor_diaria, franquia_km_diaria, descricao
)
SELECT 4, c.Id_categoria::TEXT, NULL, c.Nome_categoria, NULL,
    c.Valor_diaria_base, NULL, c.Descricao_categoria
FROM src_bd_dw_26_1.Categoria c;

-- stg_veiculo (todos no patio dono)
INSERT INTO staging.stg_veiculo (
    sk_fonte, id_natural, placa, chassi, renavam, marca, modelo,
    cor,
    ano_fabricacao, mecanizacao, tem_ar_condicionado,
    capacidade_pessoas, capacidade_porta_malas,
    categoria_dimensoes,
    km_atual, situacao, grupo_id_natural, patio_id_natural
)
SELECT 4, v.Id_veiculo::TEXT, v.Placa, v.Chassi, NULL,
    v.Marca, v.Modelo, v.Cor, v.Ano,
    CASE WHEN v.Tipo_cambio ILIKE '%automatic%' THEN 'AUTOMATICA'
        WHEN v.Tipo_cambio ILIKE '%manual%' THEN 'MANUAL'

```

```

        ELSE 'DESCONHECIDA' END,
v.Possui_ar_condicionado, v.Capacidade_pessoas,
v.Capacidade_porta_malas, v.Dimensoes, v.Km_atual,
v.Status_veiculo,
v.Id_categoria::TEXT,
CASE WHEN v.Id_empresa = 1 THEN '1' ELSE v.Id_empresa::TEXT
END
FROM src_bd_dw_26_1.Veiculo v;

-- stg_cliente
INSERT INTO staging.stg_cliente (
    sk_fonte, id_natural, tipo_pessoa, nome, nome_fantasia,
    cidade_origem, uf_origem, email, telefone, cpf, cnpj,
    data_nascimento, flag_tem_condutor_associado
)
SELECT 4, c.Id_cliente::TEXT, c.Tipo_cliente,
COALESCE(pf.Nome_cliente, pj.Razao_social), pj.Nome_fantasia,
e.Cidade, e.Uf, c.Email_cliente, c.Telefone_cliente,
pf.Cpf_cliente, pj.Cnpj_cliente, pf.Data_nascimento_cliente,
EXISTS (SELECT 1 FROM src_bd_dw_26_1.Motorista m
        WHERE m.Id_cliente = c.Id_cliente)
FROM src_bd_dw_26_1.Cliente c
LEFT JOIN src_bd_dw_26_1.Cliente_pf pf ON pf.Id_cliente =
c.Id_cliente
LEFT JOIN src_bd_dw_26_1.Cliente_pj pj ON pj.Id_cliente =
c.Id_cliente
LEFT JOIN src_bd_dw_26_1.Endereco e ON e.Id_endereco =
c.Id_endereco;

-- stg_reserva (filtra stubs > 100)
INSERT INTO staging.stg_reserva (
    sk_fonte, id_natural, cliente_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_reserva, data_retirada_prevista, data_devolucao_prevista,
    qtd_veiculos_solicitados, valor_previsto, status_origem
)
SELECT 4, r.Id_reserva::TEXT, r.Id_cliente::TEXT,
r.Id_categoria::TEXT,
r.Id_patio_previsto_retirada::TEXT,
r.Id_patio_previsto_devolucao::TEXT,
r.Data_hora_reserva, r.Data_previsao_retirada,
r.Data_previsao_devolucao,
1, r.Valor_previsto, r.Status_reserva
FROM src_bd_dw_26_1.Reserva r
WHERE r.Id_reserva <= 100;

-- stg_locacao

```

```

INSERT INTO staging.stg_locacao (
    sk_fonte, id_natural, numero_contrato, reserva_id_natural,
    cliente_id_natural, veiculo_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_retirada_real, data_devolucao_real,
        data_devolucao_prevista,
    km_saida, km_chegada, valor_diaria_aplicada,
        valor_total_final,
    status_origem
)
SELECT 4, l.Id_locacao::TEXT, NULL, l.Id_reserva::TEXT,
    m.Id_cliente::TEXT, l.Id_veiculo::TEXT, v.Id_categoria::TEXT,
    l.Id_patio_real_retirada::TEXT,
    CASE WHEN l.Data_hora_devolucao_real IS NOT NULL
        THEN l.Id_patio_real_devolucao::TEXT END,
    l.Data_hora_retirada_real, l.Data_hora_devolucao_real,
    r_stub.Data_previsao_devolucao,
    l.Km_retirada, l.Km_devolucao, NULL, l.Valor_total_final,
    l.Status_locacao
FROM      src_bd_dw_26_1.Locacao      l
LEFT JOIN src_bd_dw_26_1.Veiculo      v      ON v.Id_veiculo =
    l.Id_veiculo
LEFT JOIN src_bd_dw_26_1.Motorista    m      ON m.Id_motorista =
    l.Id_motorista
LEFT JOIN src_bd_dw_26_1.Reserva      r_stub ON r_stub.Id_reserva
    = l.Id_reserva;

RESET search_path;

```

11.5 A.5 etl/05_extract_bigdata.sql

```

-- =====
--  Avaliacao 02 – Modelagem de Data Warehouse – Parte II
--  Arquivo: etl/05_extract_bigdata.sql
--  Grupo:
--      - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--      - Andre Vinicius Lobo Giron          – DRE 122050404
--
--  Particularidades:
--      - Cliente: UNION ALL (PessoaFisica + Empresa).
--      - Reserva sem grupo (IDCategoria nao existe) -> sk_grupo = 0
--        (P-10).
--      - Locacao: patio_retirada via Vaga.IDPatio.
--  =====

```

```

SET search_path = staging, public;

```

```

DELETE FROM staging.stg_movimentacao_patio WHERE sk_fonte = 5;
DELETE FROM staging.stg_locacao WHERE sk_fonte = 5;
DELETE FROM staging.stg_reserva WHERE sk_fonte = 5;
DELETE FROM staging.stg_cliente WHERE sk_fonte = 5;
DELETE FROM staging.stg_veiculo WHERE sk_fonte = 5;
DELETE FROM staging.stg_grupo WHERE sk_fonte = 5;
DELETE FROM staging.stg_patio WHERE sk_fonte = 5;

-- stg_patio
INSERT INTO staging.stg_patio (
    sk_fonte, id_natural, nome_original, endereco, cidade,
    tipo_local,
    capacidade_vagas, funciona_24h
)
SELECT 5, p.IDPatio::TEXT, p.CDPatio,
    e.Logradouro || ', ' || e.Numero || COALESCE(' - ' ||
        e.Bairro, ''),
    e.Cidade,
    CASE WHEN p.CDPatio ILIKE '%GIG%' OR p.CDPatio ILIKE '%SDU%'
        THEN 'AEROPORTO'
        WHEN p.CDPatio ILIKE '%ROD%'
        THEN 'RODOVIARIA'
        WHEN p.CDPatio ILIKE '%NAM%' OR p.CDPatio ILIKE '%RSL%'
        OR p.CDPatio ILIKE '%BRR%'
        THEN 'SHOPPING'
        ELSE 'DESCONHECIDO' END,
    p.Lotacao,
    (p.HorarioAbertura = TIME '00:00' AND p.HorarioFechamento >=
        TIME '23:59')
FROM src_bigdata.Patio p
LEFT JOIN src_bigdata.Endereco e ON e.IDEndereco = p.IDEndereco;

-- stg_grupo (Categoria)
INSERT INTO staging.stg_grupo (
    sk_fonte, id_natural, codigo_origem, nome_origem, classe_luxo,
    valor_diaria, franquia_km_diaria, descricao
)
SELECT 5, c.IDCategoria::TEXT, NULL, c.Classificacao,
    CASE c.ClasseLuxo WHEN 'A' THEN 'LUXO'
        WHEN 'B' THEN 'INTERMEDIARIO'
        WHEN 'C' THEN 'ECONOMICO'
        ELSE 'DESCONHECIDA' END,
    c.ValorDiariaBase, NULL, c.Classificacao
FROM src_bigdata.Categoria c;

```

```

-- stg_veiculo (todos no patio dono)
INSERT INTO staging.stg_veiculo (
    sk_fonte, id_natural, placa, chassi, renavam, marca, modelo,
    cor,
    ano_fabricacao, mecanizacao, tem_ar_condicionado,
    tem_cadeira_infantil,
    km_atual, situacao, grupo_id_natural, patio_id_natural
)
SELECT 5, v.IDVeiculo::TEXT, v.Placa, v.Chassi, NULL,
    NULL, v.Modelo, NULL, v.Ano, 'DESCONHECIDA',
    v.ArCondicionado, v.CadeiraInfantil, v.UltimaKilometragem,
    'DISPONIVEL', v.IDCategoria::TEXT, '1'
FROM src_bigdata.Veiculo v;

-- stg_cliente (UNION ALL PF + PJ)
INSERT INTO staging.stg_cliente (
    sk_fonte, id_natural, tipo_pessoa, nome, nome_fantasia,
    cidade_origem, uf_origem, email, telefone, cpf, cnpj,
    data_nascimento, flag_tem_condutor_associado
)
SELECT 5, 'PF_' || pf.IDFisica, 'PF', pf.Nome, NULL,
    e.Cidade, e.UF, NULL, pf.Telefone, pf.CPF, NULL,
    pf.DtNascimento, TRUE
FROM src_bigdata.PessoaFisica pf
LEFT JOIN src_bigdata.Endereco e ON e.IDEndereco = pf.IDEndereco
UNION ALL
SELECT 5, 'PJ_' || emp.IDEmpresa, 'PJ', emp.RazaoSocial, NULL,
    e.Cidade, e.UF, NULL, emp.Telefone, NULL, emp.CNPJ, NULL,
    FALSE
FROM src_bigdata.Empresa emp
LEFT JOIN src_bigdata.Endereco e ON e.IDEndereco = emp.IDEndereco;

-- stg_reserva (sem grupo - P-10)
INSERT INTO staging.stg_reserva (
    sk_fonte, id_natural, cliente_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_reserva, data_retirada_prevista, data_devolucao_prevista,
    qtd_veiculos_solicitados, valor_previsto, status_origem
)
SELECT 5, r.IDReserva::TEXT,
    CASE WHEN cc.IDFisica IS NOT NULL THEN 'PF_' || cc.IDFisica
    WHEN cc.IDEmpresa IS NOT NULL THEN 'PJ_' || cc.IDEmpresa
    END,
    NULL, '1', '1',
    r.DtReserva, r.DtRetiradaPrevista, r.DtLimiteRetirada,
    r.QtVeiculosSolicitados, NULL, r.Status

```

```

FROM      src_bigdata.Reserva      r
LEFT JOIN src_bigdata.CentroCusto cc ON cc.IDCentroCusto =
        r.IDCentroCusto
WHERE r.IDReserva <= 100;

-- stg_locacao (patio via Vaga)
INSERT INTO staging.stg_locacao (
    sk_fonte, id_natural, numero_contrato, reserva_id_natural,
    cliente_id_natural, veiculo_id_natural, grupo_id_natural,
    patio_retirada_id_natural, patio_devolucao_id_natural,
    data_retirada_real, data_devolucao_real,
    data_devolucao_prevista,
    km_saida, km_chegada, valor_diaria_aplicada,
    valor_total_final,
    status_origem
)
SELECT 5, l.IDLocacao::TEXT, NULL, l.IDReserva::TEXT,
    'PF_' || mot.IDFisica, l.IDVeiculo::TEXT, v.IDCategoria::TEXT,
    vag_ret.IDPatio::TEXT,
    CASE WHEN l.DtChegada IS NOT NULL THEN vag_dev.IDPatio::TEXT
    END,
    l.DtRetirada, l.DtChegada,
    l.DtRetirada + INTERVAL '5 days',
    NULL, NULL, l.ValorDiaria, NULL,
    CASE WHEN l.DtChegada IS NOT NULL THEN 'CONCLUIDA' ELSE
    'EM_ANDAMENTO' END
FROM      src_bigdata.Locacao      l
LEFT JOIN src_bigdata.Veiculo      v      ON v.IDVeiculo =
        l.IDVeiculo
LEFT JOIN src_bigdata.Motorista    mot    ON mot.IDMotorista =
        l.IDMotorista
LEFT JOIN src_bigdata.Vaga          vag_ret ON vag_ret.IDVaga =
        l.IDVagaRetirada
LEFT JOIN src_bigdata.Vaga          vag_dev ON vag_dev.IDVaga =
        l.IDVagaDevolvida;

-- stg_movimentacao_patio (via Vagas)
INSERT INTO staging.stg_movimentacao_patio (
    sk_fonte, id_natural, veiculo_id_natural,
    patio_origem_id_natural, patio_destino_id_natural,
    data_movimentacao, motivo
)
SELECT 5, m.IDMovimentacao::TEXT, m.IDVeiculo::TEXT,
    vag_o.IDPatio::TEXT, vag_d.IDPatio::TEXT,
    m.DtRetirada, 'Movimentacao via vaga'
FROM      src_bigdata.Movimentacao m
LEFT JOIN src_bigdata.Vaga vag_o ON vag_o.IDVaga = m.IDVagaOrigem

```

```
LEFT JOIN src_bigdata.Vaga vag_d ON vag_d.IDVaga =  
m.IDVagaDestino;
```

```
RESET search_path;
```

12 Apêndice B — Script de Transform (etl/06_transform.sql)

```
-- =====  
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II  
-- Arquivo: etl/06_transform.sql  
-- Grupo:  
--   - Gustavo Oliveira Pessanha da Silva – DRE 122051824  
--   - Andre Vinicius Lobo Giron          – DRE 122050404  
-- =====  
  
SET search_path = staging, public;  
  
DO $$  
DECLARE  
    v_count INTEGER;  
BEGIN  
    RAISE NOTICE '==== Transform START ====';  
  
    -- 1. NORMALIZACAO DE TEXTO (UPPER + TRIM)  
    UPDATE staging.stg_patio  
        SET nome_original = TRIM(nome_original),  
            cidade         = UPPER(TRIM(COALESCE(cidade, '')));  
  
    UPDATE staging.stg_grupo  
        SET nome_origem = UPPER(TRIM(COALESCE(nome_origem, ''))),  
            classe_luxo = UPPER(TRIM(COALESCE(classe_luxo, '')));  
  
    UPDATE staging.stg_cliente  
        SET nome           = TRIM(COALESCE(nome, '')),  
            cidade_origem = UPPER(TRIM(COALESCE(cidade_origem,  
            '')));  
  
    UPDATE staging.stg_veiculo  
        SET marca          = UPPER(TRIM(COALESCE(marca, ''))),  
            modelo         = UPPER(TRIM(COALESCE(modelo, ''))),  
            cor             = UPPER(TRIM(COALESCE(cor, ''))),
```

```

        mecanizacao = UPPER(TRIM(COALESCE(mecanizacao, ''))),
        situacao     = UPPER(TRIM(COALESCE(situacao, '')));

-- 2. APLICAR depara_patio em stg_patio, stg_veiculo,
--    stg_reserva (retirada+devolucao), stg_locacao (idem),
--    stg_movimentacao_patio (origem+destino).
UPDATE staging.stg_patio sp
    SET nome_canonico = dp.nome_canonico
    FROM staging.depara_patio dp
    WHERE dp.sk_fonte          = sp.sk_fonte
    AND dp.id_natural_origem = sp.id_natural;

UPDATE staging.stg_veiculo sv
    SET nome_canonico_patio = dp.nome_canonico
    FROM staging.depara_patio dp
    WHERE dp.sk_fonte          = sv.sk_fonte
    AND dp.id_natural_origem = sv.patio_id_natural;

UPDATE staging.stg_reserva sr
    SET nome_canonico_patio_retirada = dp.nome_canonico
    FROM staging.depara_patio dp
    WHERE dp.sk_fonte          = sr.sk_fonte
    AND dp.id_natural_origem = sr.patio_retirada_id_natural;

UPDATE staging.stg_reserva sr
    SET nome_canonico_patio_devolucao = dp.nome_canonico
    FROM staging.depara_patio dp
    WHERE dp.sk_fonte          = sr.sk_fonte
    AND dp.id_natural_origem = sr.patio_devolucao_id_natural;

UPDATE staging.stg_locacao sl
    SET nome_canonico_patio_retirada = dp.nome_canonico
    FROM staging.depara_patio dp
    WHERE dp.sk_fonte          = sl.sk_fonte
    AND dp.id_natural_origem = sl.patio_retirada_id_natural;

UPDATE staging.stg_locacao sl
    SET nome_canonico_patio_devolucao = dp.nome_canonico
    FROM staging.depara_patio dp
    WHERE dp.sk_fonte          = sl.sk_fonte
    AND dp.id_natural_origem = sl.patio_devolucao_id_natural;

UPDATE staging.stg_movimentacao_patio sm
    SET nome_canonico_origem = dp.nome_canonico

```

```

    FROM staging.depara_patio dp
  WHERE dp.sk_fonte          = sm.sk_fonte
        AND dp.id_natural_origem = sm.patio_origem_id_natural;

UPDATE staging.stg_movimentacao_patio sm
  SET nome_canonico_destino = dp.nome_canonico
  FROM staging.depara_patio dp
  WHERE dp.sk_fonte          = sm.sk_fonte
        AND dp.id_natural_origem = sm.patio_destino_id_natural;

-- 3. APLICAR depara_grupo em stg_grupo, stg_veiculo,
--    stg_reserva, stg_locacao.
UPDATE staging.stg_grupo sg
  SET nome_canonico = dg.nome_canonico
  FROM staging.depara_grupo dg
  WHERE dg.sk_fonte      = sg.sk_fonte
        AND dg.codigo_origem = sg.id_natural;

UPDATE staging.stg_veiculo sv
  SET nome_canonico_grupo = dg.nome_canonico
  FROM staging.depara_grupo dg
  WHERE dg.sk_fonte      = sv.sk_fonte
        AND dg.codigo_origem = sv.grupo_id_natural;

UPDATE staging.stg_reserva sr
  SET nome_canonico_grupo = dg.nome_canonico
  FROM staging.depara_grupo dg
  WHERE dg.sk_fonte      = sr.sk_fonte
        AND dg.codigo_origem = sr.grupo_id_natural;

UPDATE staging.stg_locacao sl
  SET nome_canonico_grupo = dg.nome_canonico
  FROM staging.depara_grupo dg
  WHERE dg.sk_fonte      = sl.sk_fonte
        AND dg.codigo_origem = sl.grupo_id_natural;

-- 4. NORMALIZAR STATUS
UPDATE staging.stg_locacao
  SET status_normalizado = CASE
    WHEN UPPER(TRIM(status_origem))
      IN
      ('EM_ANDAMENTO', 'ATIVA')
    THEN 'EM_ANDAMENTO'
    WHEN UPPER(TRIM(status_origem))

```

```

        IN
        ('CONCLUÍDA', 'FINALIZADA', 'CONCLUÍDA')
    THEN 'CONCLUÍDA'
        WHEN UPPER(TRIM(status_origem))
        IN
        ('CANCELADA', 'CANCELED', 'CANCELADO')
    THEN 'CANCELADA'
        ELSE 'DESCONHECIDO' END;

UPDATE staging.stg_reserva
SET status_normalizado = CASE
    WHEN UPPER(TRIM(status_origem))
    IN
    ('CONFIRMADA', 'ATIVA', 'CONFIRMED')
    THEN 'CONFIRMADA'
    WHEN UPPER(TRIM(status_origem))
    IN
    ('EM_FILA_ESPERA', 'ESPERA', 'WAITING')
    THEN 'EM_FILA_ESPERA'
    WHEN UPPER(TRIM(status_origem))
    IN
    ('CANCELADA', 'CANCELED', 'CANCELADO')
    THEN 'CANCELADA'
    WHEN UPPER(TRIM(status_origem))
    IN
    ('CONCRETIZADA', 'ATENDIDA', 'CONVERTIDA')
    THEN 'CONCRETIZADA'
    ELSE 'DESCONHECIDO' END;

-- 5. DERIVAR duracao_real_dias / km_rodados
UPDATE staging.stg_locacao
SET duracao_real_dias = CASE
    WHEN data_devolucao_real IS NOT NULL AND
    data_retirada_real IS NOT NULL
    THEN EXTRACT(EPOCH FROM (data_devolucao_real -
    data_retirada_real))::INTEGER
        / 86400
    ELSE NULL END,
    km_rodados = CASE
    WHEN km_chegada IS NOT NULL AND km_saida IS NOT NULL
        AND km_chegada >= km_saida
    THEN km_chegada - km_saida
    ELSE NULL END;

-- 6. CIDADE NULL -> 'CIDADE_DESCONHECIDA'
UPDATE staging.stg_cliente
SET cidade_origem = 'CIDADE_DESCONHECIDA'
WHERE cidade_origem IS NULL OR cidade_origem = '';

```

```

-- 7. LOGS DE QUALIDADE
SELECT COUNT(*) INTO v_count FROM staging.stg_locacao
  WHERE nome_canonico_patio_retirada IS NULL;
IF v_count > 0 THEN
  RAISE NOTICE 'AVISO: % linhas stg_locacao SEM
    nome_canonico_patio_retirada.', v_count;
END IF;

SELECT COUNT(*) INTO v_count FROM staging.stg_reserva
  WHERE nome_canonico_patio_retirada IS NULL;
IF v_count > 0 THEN
  RAISE NOTICE 'AVISO: % linhas stg_reserva SEM
    nome_canonico_patio_retirada.', v_count;
END IF;

SELECT COUNT(*) INTO v_count FROM staging.stg_veiculo
  WHERE nome_canonico_patio IS NULL;
IF v_count > 0 THEN
  RAISE NOTICE 'AVISO: % linhas stg_veiculo SEM
    nome_canonico_patio.', v_count;
END IF;

SELECT COUNT(*) INTO v_count FROM staging.stg_reserva
  WHERE sk_fonte = 5 AND nome_canonico_grupo IS NULL;
RAISE NOTICE 'INFO: % reservas bigdata sem grupo (P-10 – irao
  para sk_grupo=0).', v_count;

RAISE NOTICE '==== Transform DONE ====';
END $$;

RESET search_path;

```

13 Apêndice C — Scripts de Load

13.1 C.1 etl/07_load_dimensoes.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: etl/07_load_dimensoes.sql
-- Grupo:
--   - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--   - Andre Vinicius Lobo Giron – DRE 122050404

```

```
-- Ordem: dim_fonte -> dim_patio -> dim_grupo -> dim_veiculo ->
      dim_cliente.
```

```
-- =====
```

```
SET search_path = dw, staging, public;
```

```
-- 1. dim_fonte
```

```
INSERT INTO dw.dim_fonte (sk_fonte, codigo_fonte,
      nome_empresa_associada, sgbd_original, descricao) VALUES
      (1, 'andre_gustavo', 'Grupo Gustavo + Andre',
        'POSTGRES',
        'Fonte Parte I do proprio grupo; Postgres nativo.'),
      (2, 'mae016', 'Grupo MAE016 (Breno, Hygor, Joao)',
        'MYSQL',
        'Sistema MAE016 traduzido de MySQL para Postgres.'),
      (3, 'locadora_db', 'Grupo Locadora-DB (Tadeu, Vicente)',
        'POSTGRES',
        'Sistema Locadora-DB; veiculo->patio extensao P-09.'),
      (4, 'bd_dw_26_1', 'Grupo BD-DW-26.1 (Ana, Mariana, +)',
        'MYSQL',
        'Sistema BD-DW traduzido de MySQL para Postgres.'),
      (5, 'bigdata', 'Grupo BigData',
        'ANSI',
        'Sistema BigData ANSI. Reserva sem categoria (P-10).')
```

```
ON CONFLICT (sk_fonte) DO UPDATE SET
```

```
      codigo_fonte          = EXCLUDED.codigo_fonte,
      nome_empresa_associada = EXCLUDED.nome_empresa_associada,
      sgbd_original         = EXCLUDED.sgbd_original,
      descricao             = EXCLUDED.descricao;
```

```
-- 2. dim_patio (consolida 6 canonicos a partir do depara_patio)
```

```
INSERT INTO dw.dim_patio (
      nome_canonico, apelido, tipo_local, cidade,
      endereco_descritivo,
      capacidade_vagas_referencia, flag_funciona_24h,
      codigo_fonte_dona
)
```

```
SELECT p.nome_canonico,
```

```
      CASE p.nome_canonico
```

```
      WHEN 'Aeroporto do Galeao' THEN 'Galeao'
```

```
      WHEN 'Aeroporto Santos Dumont' THEN 'Santos Dumont'
```

```
      WHEN 'Rodoviaria do Rio' THEN 'Rodoviaria'
```

```
      WHEN 'Shopping Rio Sul' THEN 'Rio Sul'
```

```
      WHEN 'Shopping Nova America' THEN 'Nova America'
```

```
      WHEN 'Barra Shopping' THEN 'Barra'
```

```
      ELSE p.nome_canonico END,
```

```
      CASE WHEN p.nome_canonico ILIKE '%Aeroporto%' THEN
        'AEROPORTO'
```

```

        WHEN p.nome_canonico ILIKE '%Rodoviaria%' THEN
        'RODOVIARIA'
        WHEN p.nome_canonico ILIKE '%Shopping%' THEN 'SHOPPING'
        WHEN p.nome_canonico ILIKE '%Barra%' THEN 'SHOPPING'
        ELSE 'DESCONHECIDO' END,
    'Rio de Janeiro',
    (SELECT MAX(sp.endereco) FROM staging.stg_patio sp
     WHERE sp.nome_canonico = p.nome_canonico),
    (SELECT MAX(sp.capacidade_vagas) FROM staging.stg_patio sp
     WHERE sp.nome_canonico = p.nome_canonico),
    COALESCE((SELECT BOOL_OR(sp.funciona_24h) FROM
    staging.stg_patio sp
             WHERE sp.nome_canonico = p.nome_canonico), FALSE),
    CASE p.nome_canonico
        WHEN 'Aeroporto do Galeao' THEN 'andre_gustavo'
        WHEN 'Aeroporto Santos Dumont' THEN 'mae016'
        WHEN 'Rodoviaria do Rio' THEN 'locadora_db'
        WHEN 'Shopping Rio Sul' THEN 'bd_dw_26_1'
        WHEN 'Shopping Nova America' THEN 'bigdata'
        WHEN 'Barra Shopping' THEN NULL END
FROM (SELECT DISTINCT nome_canonico FROM staging.depara_patio) p
ORDER BY p.nome_canonico
ON CONFLICT (nome_canonico) DO UPDATE SET
    apelido = EXCLUDED.apelido, tipo_local = EXCLUDED.tipo_local,
    cidade = EXCLUDED.cidade, endereco_descritivo =
        EXCLUDED.endereco_descritivo,
    capacidade_vagas_referencia =
        EXCLUDED.capacidade_vagas_referencia,
    flag_funciona_24h = EXCLUDED.flag_funciona_24h,
    codigo_fonte_dona = EXCLUDED.codigo_fonte_dona;

-- 3. dim_grupo (classe_luxo via MODE, valor_diaria via AVG nas 4
fontes)
INSERT INTO dw.dim_grupo (
    nome_grupo_normalizado, codigo_curto, classe_luxo,
    valor_diaria_referencia, franquia_km_diaria_referencia,
    descricao
)
SELECT g.nome_canonico, NULL,
    COALESCE(
        (SELECT CASE UPPER(TRIM(sg2.classe_luxo))
            WHEN 'LUXO' THEN 'LUXO'
            WHEN 'INTERMEDIARIO' THEN 'INTERMEDIARIO'
            WHEN 'ECONOMICO' THEN 'ECONOMICO'
            ELSE 'DESCONHECIDA' END
        FROM staging.stg_grupo sg2

```

```

        JOIN staging.depara_grupo dg2
            ON dg2.sk_fonte = sg2.sk_fonte AND
            dg2.codigo_origem = sg2.id_natural
        WHERE dg2.nome_canonico = g.nome_canonico
            AND sg2.classe_luxo IS NOT NULL AND
            TRIM(sg2.classe_luxo) <> ''
        GROUP BY UPPER(TRIM(sg2.classe_luxo))
        ORDER BY COUNT(*) DESC LIMIT 1), 'DESCONHECIDA'),
    (SELECT AVG(sg2.valor_diaria) FROM staging.stg_grupo sg2
    JOIN staging.depara_grupo dg2
        ON dg2.sk_fonte = sg2.sk_fonte AND dg2.codigo_origem
        = sg2.id_natural
    WHERE dg2.nome_canonico = g.nome_canonico AND
        sg2.valor_diaria IS NOT NULL),
    NULL, g.nome_canonico
FROM (SELECT DISTINCT nome_canonico FROM staging.depara_grupo) g
ORDER BY g.nome_canonico
ON CONFLICT (nome_grupo_normalizado) DO UPDATE SET
    classe_luxo = EXCLUDED.classe_luxo,
    valor_diaria_referencia = EXCLUDED.valor_diaria_referencia,
    descricao = EXCLUDED.descricao;

-- 4. dim_veiculo
INSERT INTO dw.dim_veiculo (
    sk_fonte_origem, id_natural_origem, placa, chassi, renavam,
    marca, modelo, cor, ano_fabricacao, mecanizacao,
    tem_ar_condicionado, tem_adaptacao_cadeirante,
    capacidade_pessoas, capacidade_porta_malas,
    categoria_dimensoes,
    situacao_atual
)
SELECT sv.sk_fonte, sv.id_natural, sv.placa, sv.chassi,
    sv.renavam,
    COALESCE(NULLIF(sv.marca, ''), 'DESCONHECIDA'),
    COALESCE(NULLIF(sv.modelo, ''), 'DESCONHECIDO'),
    sv.cor, sv.ano_fabricacao,
    CASE sv.mecanizacao WHEN 'MANUAL' THEN 'MANUAL'
        WHEN 'AUTOMATICA' THEN 'AUTOMATICA'
        ELSE 'DESCONHECIDA' END,
    sv.tem_ar_condicionado, sv.tem_cadeira_infantil,
    sv.capacidade_pessoas, sv.capacidade_porta_malas,
    sv.categoria_dimensoes,
    CASE WHEN sv.situacao
        IN
        ('DISPONIVEL', 'ALUGADO', 'MANUTENCAO', 'BAIXADO', 'RESERVADO')
        THEN sv.situacao ELSE 'DESCONHECIDA' END
FROM staging.stg_veiculo sv

```

```

ON CONFLICT (sk_fonte_origem, id_natural_origem) DO UPDATE SET
    placa = EXCLUDED.placa, chassi = EXCLUDED.chassi, renavam =
        EXCLUDED.renavam,
    marca = EXCLUDED.marca, modelo = EXCLUDED.modelo, cor =
        EXCLUDED.cor,
    ano_fabricacao = EXCLUDED.ano_fabricacao, mecanizacao =
        EXCLUDED.mecanizacao,
    tem_ar_condicionado = EXCLUDED.tem_ar_condicionado,
    tem_adaptacao_cadeirante = EXCLUDED.tem_adaptacao_cadeirante,
    capacidade_pessoas = EXCLUDED.capacidade_pessoas,
    capacidade_porta_malas = EXCLUDED.capacidade_porta_malas,
    categoria_dimensoes = EXCLUDED.categoria_dimensoes,
    situacao_atual = EXCLUDED.situacao_atual;

-- 5. dim_cliente
INSERT INTO dw.dim_cliente (
    sk_fonte_origem, id_natural_origem, tipo_pessoa, nome,
    nome_fantasia,
    cidade_origem, uf_origem, email, telefone, cpf_normalizado,
    cnpj_normalizado, flag_eh_pessoa_juridica
)
SELECT sc.sk_fonte, sc.id_natural,
    CASE WHEN sc.tipo_pessoa IN ('PF','PJ') THEN sc.tipo_pessoa
    ELSE 'PF' END,
    COALESCE(NULLIF(sc.nome, ''), 'CLIENTE DESCONHECIDO'),
    sc.nome_fantasia,
    COALESCE(NULLIF(sc.cidade_origem, ''), 'DESCONHECIDA'),
    sc.uf_origem, sc.email, sc.telefone, sc.cpf, sc.cnpj,
    (sc.tipo_pessoa = 'PJ')
FROM staging.stg_cliente sc
ON CONFLICT (sk_fonte_origem, id_natural_origem) DO UPDATE SET
    tipo_pessoa = EXCLUDED.tipo_pessoa, nome = EXCLUDED.nome,
    nome_fantasia = EXCLUDED.nome_fantasia, cidade_origem =
        EXCLUDED.cidade_origem,
    uf_origem = EXCLUDED.uf_origem, email = EXCLUDED.email,
    telefone = EXCLUDED.telefone, cpf_normalizado =
        EXCLUDED.cpf_normalizado,
    cnpj_normalizado = EXCLUDED.cnpj_normalizado,
    flag_eh_pessoa_juridica = EXCLUDED.flag_eh_pessoa_juridica;

RESET search_path;

```

13.2 C.2 etl/08_load_fatos.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II

```

```
-- Arquivo: etl/08_load_fatos.sql
-- Grupo:
--   - Gustavo Oliveira Pessanha da Silva – DRE 122051824
--   - Andre Vinicius Lobo Giron          – DRE 122050404
--
-- Ordem: fato_locacao -> fato_reserva -> fato_patio_diario.
-- Trata CRITICO-01 (canceladas fonte 3) e CRITICO-02 (LATERAL
--   determinista).
-- =====
```

```
SET search_path = dw, staging, public;
```

```
-- 0. Pre-condicao
```

```
DO $$
```

```
BEGIN
```

```
  IF (SELECT COUNT(*) FROM staging.stg_veiculo) = 0 THEN
    RAISE EXCEPTION '08_load_fatos requer 01-05 + 06
    executados (stg_veiculo vazia).';
```

```
  END IF;
```

```
END $$;
```

```
-- 1. fato_locacao
```

```
TRUNCATE TABLE dw.fato_locacao RESTART IDENTITY;
```

```
INSERT INTO dw.fato_locacao (
  sk_tempo_retirada_real, sk_tempo_devolucao_real,
  sk_tempo_devolucao_prevista,
  sk_patio_retirada, sk_patio_devolucao,
  sk_veiculo, sk_grupo, sk_cliente, sk_fonte,
  id_locacao_origem, numero_contrato_fonte, status_locacao,
  qtd_locacoes, duracao_prevista_dias, duracao_real_dias,
  km_rodados,
  valor_diaria_aplicada, valor_total_estimado,
  valor_total_final,
  sk_fonte_id_natural
)
```

```
)
```

```
SELECT
```

```
  COALESCE(
    (EXTRACT(YEAR FROM sl.data_retirada_real) * 10000
     + EXTRACT(MONTH FROM sl.data_retirada_real) * 100
     + EXTRACT(DAY FROM sl.data_retirada_real))::INTEGER,
    19000101),
    CASE WHEN sl.status_normalizado = 'EM_ANDAMENTO'
      OR sl.data_devolucao_real IS NULL THEN NULL
    ELSE (EXTRACT(YEAR FROM sl.data_devolucao_real) * 10000
     + EXTRACT(MONTH FROM sl.data_devolucao_real) * 100
```

```

        + EXTRACT(DAY FROM
sl.data_devolucao_real))::INTEGER END,
COALESCE(
    (EXTRACT(YEAR FROM sl.data_devolucao_prevista) * 10000
    + EXTRACT(MONTH FROM sl.data_devolucao_prevista) * 100
    + EXTRACT(DAY FROM sl.data_devolucao_prevista))::INTEGER,
    (EXTRACT(YEAR FROM sl.data_retirada_real) * 10000
    + EXTRACT(MONTH FROM sl.data_retirada_real) * 100
    + EXTRACT(DAY FROM sl.data_retirada_real))::INTEGER,
    19000101),
dp_ret.sk_patio,
CASE WHEN sl.data_devolucao_real IS NULL THEN NULL
      ELSE dp_dev.sk_patio END,
dv.sk_veiculo, COALESCE(dg.sk_grupo, 0), dc.sk_cliente,
sl.sk_fonte, sl.id_natural::BIGINT, sl.numero_contrato,
sl.status_normalizado, 1,
CASE WHEN sl.data_devolucao_prevista IS NOT NULL AND
sl.data_retirada_real IS NOT NULL
      THEN EXTRACT(EPOCH FROM (sl.data_devolucao_prevista -
sl.data_retirada_real))::INTEGER
        / 86400
      ELSE NULL END,
sl.duracao_real_dias, sl.km_rodados, sl.valor_diaria_aplicada,
CASE WHEN sl.valor_diaria_aplicada IS NOT NULL
      AND sl.data_devolucao_prevista IS NOT NULL
      AND sl.data_retirada_real IS NOT NULL
      AND (sl.data_devolucao_prevista::DATE -
sl.data_retirada_real::DATE) > 0
      THEN sl.valor_diaria_aplicada
        * (sl.data_devolucao_prevista::DATE -
sl.data_retirada_real::DATE)
      ELSE NULL END,
sl.valor_total_final, sl.id_natural
FROM      staging.stg_locacao sl
LEFT JOIN dw.dim_patio   dp_ret ON dp_ret.nome_canonico =
      sl.nome_canonico_patio_retirada
LEFT JOIN dw.dim_patio   dp_dev ON dp_dev.nome_canonico =
      sl.nome_canonico_patio_devolucao
LEFT JOIN dw.dim_veiculo dv      ON dv.sk_fonte_origem =
      sl.sk_fonte
                        AND dv.id_natural_origem =
      sl.veiculo_id_natural
LEFT JOIN dw.dim_grupo   dg      ON dg.nome_grupo_normalizado =
      sl.nome_canonico_grupo
LEFT JOIN dw.dim_cliente dc     ON dc.sk_fonte_origem =
      sl.sk_fonte
                        AND dc.id_natural_origem =
      sl.cliente_id_natural
WHERE sl.status_normalizado <> 'DESCONHECIDO'

```

```

AND dp_ret.sk_patio    IS NOT NULL
AND dv.sk_veiculo     IS NOT NULL
AND dc.sk_cliente     IS NOT NULL
ON CONFLICT (sk_fonte, sk_fonte_id_natural) DO NOTHING;

-- 2. fato_reserva
TRUNCATE TABLE dw.fato_reserva RESTART IDENTITY;
INSERT INTO dw.fato_reserva (
    sk_tempo_reserva, sk_tempo_retirada_prevista,
    sk_tempo_devolucao_prevista,
    sk_patio_retirada, sk_patio_devolucao,
    sk_grupo, sk_cliente, sk_fonte,
    id_reserva_origem, status_reserva,
    qtd_reservas, qtd_veiculos_solicitados,
    duracao_prevista_dias, dias_antecedencia, valor_previsto,
    sk_fonte_id_natural
)
SELECT
    (EXTRACT(YEAR FROM sr.data_reserva) * 10000
    + EXTRACT(MONTH FROM sr.data_reserva) * 100
    + EXTRACT(DAY FROM sr.data_reserva))::INTEGER,
    (EXTRACT(YEAR FROM sr.data_retirada_prevista) * 10000
    + EXTRACT(MONTH FROM sr.data_retirada_prevista) * 100
    + EXTRACT(DAY FROM sr.data_retirada_prevista))::INTEGER,
    (EXTRACT(YEAR FROM sr.data_devolucao_prevista) * 10000
    + EXTRACT(MONTH FROM sr.data_devolucao_prevista) * 100
    + EXTRACT(DAY FROM sr.data_devolucao_prevista))::INTEGER,
    dp_ret.sk_patio, COALESCE(dp_dev.sk_patio, dp_ret.sk_patio),
    COALESCE(dg.sk_grupo, 0), dc.sk_cliente,
    sr.sk_fonte, sr.id_natural::BIGINT, sr.status_normalizado, 1,
    COALESCE(sr.qtd_veiculos_solicitados, 1),
    CASE WHEN sr.data_devolucao_prevista IS NOT NULL
        AND sr.data_retirada_prevista IS NOT NULL
        THEN EXTRACT(EPOCH FROM (sr.data_devolucao_prevista
        -
        sr.data_retirada_prevista))::INTEGER / 86400
        ELSE NULL END,
    CASE WHEN sr.data_retirada_prevista IS NOT NULL
        AND sr.data_reserva IS NOT NULL
        THEN EXTRACT(EPOCH FROM (sr.data_retirada_prevista
        - sr.data_reserva))::INTEGER /
        86400
        ELSE NULL END,
    sr.valor_previsto, sr.id_natural
FROM
    staging.stg_reserva sr

```

```

LEFT JOIN dw.dim_patio dp_ret ON dp_ret.nome_canonico =
    sr.nome_canonico_patio_retirada
LEFT JOIN dw.dim_patio dp_dev ON dp_dev.nome_canonico =
    sr.nome_canonico_patio_devolucao
LEFT JOIN dw.dim_grupo dg ON dg.nome_grupo_normalizado =
    sr.nome_canonico_grupo
LEFT JOIN dw.dim_cliente dc ON dc.sk_fonte_origem =
    sr.sk_fonte
                                AND dc.id_natural_origem =
    sr.cliente_id_natural
WHERE sr.data_reserva IS NOT NULL
    AND sr.data_retirada_prevista IS NOT NULL
    AND sr.data_devolucao_prevista IS NOT NULL
    AND dp_ret.sk_patio IS NOT NULL
    AND dc.sk_cliente IS NOT NULL
ON CONFLICT (sk_fonte, sk_fonte_id_natural) DO NOTHING;

-- 3. fato_patio_diario (CRITICO-02: ORDER BY determinista)
TRUNCATE TABLE dw.fato_patio_diario RESTART IDENTITY;

DO $$
DECLARE v_overlaps INTEGER;
BEGIN
    WITH dias AS (
        SELECT generate_series((CURRENT_DATE - INTERVAL '29
            days')::DATE,
                                CURRENT_DATE, INTERVAL '1
            day')::DATE AS dia),
        loc_intervalos AS (
            SELECT fl.sk_veiculo,
                dt_ret.data_completa AS dia_retirada,
                COALESCE(dt_dev.data_completa, CURRENT_DATE) AS
                dia_devolucao
            FROM dw.fato_locacao fl
            JOIN dw.dim_tempo dt_ret ON dt_ret.sk_tempo =
                fl.sk_tempo_retirada_real
            LEFT JOIN dw.dim_tempo dt_dev ON dt_dev.sk_tempo =
                fl.sk_tempo_devolucao_real
            WHERE fl.status_locacao IN ('EM_ANDAMENTO', 'CONCLUIDA')),
        sobreposicoes AS (
            SELECT li.sk_veiculo, d.dia
            FROM loc_intervalos li
            JOIN dias d ON d.dia >= li.dia_retirada AND d.dia <
                li.dia_devolucao
            GROUP BY li.sk_veiculo, d.dia HAVING COUNT(*) > 1)
    SELECT COUNT(*) INTO v_overlaps FROM sobreposicoes;
    RAISE NOTICE 'INFO: % sobreposicoes veiculo x dia (desempeate
        via LATERAL ORDER BY).',
        v_overlaps;

```

```

END $$;

WITH dias AS (
    SELECT generate_series((CURRENT_DATE - INTERVAL '29
        days')::DATE,
                                CURRENT_DATE, INTERVAL '1 day')::DATE
        AS dia),
veic_patio AS (
    SELECT dv.sk_veiculo, dv.sk_fonte_origem AS sk_fonte,
        dv.situacao_atual,
        COALESCE(dg.sk_grupo, 0) AS sk_grupo,
        dp_origem.sk_patio AS sk_patio_origem
    FROM      dw.dim_veiculo dv
    LEFT JOIN staging.stg_veiculo sv ON sv.sk_fonte =
        dv.sk_fonte_origem
                                AND sv.id_natural =
        dv.id_natural_origem
    LEFT JOIN staging.depara_grupo dg2 ON dg2.sk_fonte =
        sv.sk_fonte
                                AND dg2.codigo_origem =
        sv.grupo_id_natural
    LEFT JOIN dw.dim_grupo dg ON
        dg.nome_grupo_normalizado = dg2.nome_canonico
    LEFT JOIN dw.dim_patio dp_origem ON dp_origem.nome_canonico
        = sv.nome_canonico_patio
    WHERE dv.situacao_atual <> 'BAIXADO'),
loc_intervalos AS (
    SELECT fl.sk_locacao, fl.sk_veiculo, fl.sk_patio_retirada,
        dt_ret.data_completa AS dia_retirada,
        COALESCE(dt_dev.data_completa, CURRENT_DATE) AS
        dia_devolucao,
        fl.status_locacao
    FROM      dw.fato_locacao fl
    JOIN      dw.dim_tempo dt_ret ON dt_ret.sk_tempo =
        fl.sk_tempo_retirada_real
    LEFT JOIN dw.dim_tempo dt_dev ON dt_dev.sk_tempo =
        fl.sk_tempo_devolucao_real
    WHERE fl.status_locacao IN ('EM_ANDAMENTO', 'CONCLUIDA')),
veic_x_dia AS (
    SELECT vp.sk_veiculo, d.dia
    FROM veic_patio vp CROSS JOIN dias d),
veic_dia_alugado AS (
    SELECT vxd.sk_veiculo, vxd.dia,
        li.sk_patio_retirada AS sk_patio_loc, li.status_locacao
    FROM      veic_x_dia vxd
    LEFT JOIN LATERAL (
        SELECT li2.sk_patio_retirada, li2.status_locacao
        FROM loc_intervalos li2

```

```

WHERE li2.sk_veiculo = vxd.sk_veiculo
AND vxd.dia >= li2.dia_retirada
AND vxd.dia < li2.dia_devolucao
ORDER BY li2.dia_retirada ASC, li2.sk_locacao ASC
LIMIT 1) li ON TRUE)
INSERT INTO dw.fato_patio_diario (
    sk_tempo, sk_patio, sk_veiculo, sk_grupo, sk_fonte,
    situacao, flag_frota_propria_no_patio, qtd_veiculos,
    capacidade_vagas_patio
)
SELECT (EXTRACT(YEAR FROM vda.dia) * 10000
+ EXTRACT(MONTH FROM vda.dia) * 100
+ EXTRACT(DAY FROM vda.dia))::INTEGER,
COALESCE(vda.sk_patio_loc, vp.sk_patio_origem),
vda.sk_veiculo, vp.sk_grupo, vp.sk_fonte,
CASE WHEN vda.sk_patio_loc IS NOT NULL THEN 'ALUGADO'
      WHEN vp.situacao_atual = 'MANUTENCAO' THEN 'MANUTENCAO'
      WHEN vp.situacao_atual = 'RESERVADO' THEN 'RESERVADO'
      ELSE 'DISPONIVEL' END,
COALESCE(df.codigo_fonte = dp.codigo_fonte_dona, FALSE),
1, dp.capacidade_vagas_referencia
FROM      veic_dia_alugado vda
JOIN      veic_patio vp ON vp.sk_veiculo = vda.sk_veiculo
LEFT JOIN dw.dim_patio dp ON dp.sk_patio =
          COALESCE(vda.sk_patio_loc, vp.sk_patio_origem)
LEFT JOIN dw.dim_fonte df ON df.sk_fonte = vp.sk_fonte
WHERE COALESCE(vda.sk_patio_loc, vp.sk_patio_origem) IS NOT NULL;

RESET search_path;

```

14 Apêndice D — Scripts de Relatórios + Markov

14.1 D.1 relatorios/01_controle_patio.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: relatorios/01_controle_patio.sql
-- Grupo: Gustavo Oliveira Pessanha (122051824), Andre Vinicius
--        Giron (122050404)
-- Relatorio gerencial (a) do enunciado – Controle de patio.

```

```

-- =====

\pset border 2
\pset null '(null)'
\set ON_ERROR_STOP on

-- Visao 1: por patio x grupo x origem (pivot por situacao)
WITH ultimo_snapshot AS (
    SELECT MAX(sk_tempo) AS sk_tempo_max FROM
        dw.fato_patio_diario),
base AS (
    SELECT pa.apelido AS patio, gr.nome_grupo_normalizado AS
        grupo,
        CASE WHEN fpd.flag_frota_propria_no_patio
            THEN 'PRÓPRIA' ELSE 'ASSOCIADA' END AS origem_frota,
        fpd.situacao
    FROM dw.fato_patio_diario fpd
    JOIN ultimo_snapshot us ON us.sk_tempo_max = fpd.sk_tempo
    JOIN dw.dim_patio pa ON pa.sk_patio = fpd.sk_patio
    JOIN dw.dim_grupo gr ON gr.sk_grupo = fpd.sk_grupo)
SELECT patio, grupo, origem_frota,
    COUNT(*) FILTER (WHERE situacao = 'DISPONÍVEL') AS
        qtd_disponiveis,
    COUNT(*) FILTER (WHERE situacao = 'ALUGADO') AS
        qtd_alugados,
    COUNT(*) FILTER (WHERE situacao = 'MANUTENÇÃO') AS
        qtd_manutencao,
    COUNT(*) FILTER (WHERE situacao = 'RESERVADO') AS
        qtd_reservados,
    COUNT(*) AS qtd_total
FROM base GROUP BY patio, grupo, origem_frota
ORDER BY patio, grupo, origem_frota;

-- Visao 2: por patio x marca x origem (total)
-- (idem para modelo e mecanizacao – código elidido aqui por
-- brevidade;
-- ver arquivo completo no repositório.)

```

14.2 D.2 relatorios/02_controle_locacoes.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: relatorios/02_controle_locacoes.sql
-- Grupo: Gustavo Oliveira Pessanha (122051824), Andre Vinicius
--       Giron (122050404)
-- Relatorio gerencial (b) – Controle das locacoes.

```

```

-- =====

\pset border 2
\pset null '(null)'
\set ON_ERROR_STOP on

-- Visao 1: locacoes por grupo x faixa de duracao prevista
WITH base AS (
    SELECT gr.nome_grupo_normalizado AS grupo,
        CASE WHEN floc.duracao_prevista_dias BETWEEN 1 AND 3
        THEN '1-3 dias'
            WHEN floc.duracao_prevista_dias BETWEEN 4 AND 7
        THEN '4-7 dias'
            WHEN floc.duracao_prevista_dias BETWEEN 8 AND 15
        THEN '8-15 dias'
            WHEN floc.duracao_prevista_dias >= 16
        THEN '16+ dias'
            ELSE 'INDEFINIDA' END AS faixa_duracao
    FROM dw.fato_locacao floc
    JOIN dw.dim_grupo gr ON gr.sk_grupo = floc.sk_grupo)
SELECT grupo, faixa_duracao, COUNT(*) AS qtd_locacoes
FROM base GROUP BY grupo, faixa_duracao
ORDER BY grupo, faixa_duracao;

-- Visao 2: tempo restante para devolucao (somente EM_ANDAMENTO)
WITH em_andamento AS (
    SELECT gr.nome_grupo_normalizado AS grupo,
        (td.data_completa - CURRENT_DATE) AS dias_restantes
    FROM dw.fato_locacao floc
    JOIN dw.dim_grupo gr ON gr.sk_grupo = floc.sk_grupo
    JOIN dw.dim_tempo td ON td.sk_tempo =
        floc.sk_tempo_devolucao_prevista
    WHERE floc.status_locacao = 'EM_ANDAMENTO')
SELECT grupo,
    CASE WHEN dias_restantes < 0 THEN 'Atrasada'
        WHEN dias_restantes = 0 THEN 'Hoje'
        WHEN dias_restantes BETWEEN 1 AND 3 THEN '1-3 dias'
        WHEN dias_restantes BETWEEN 4 AND 7 THEN '4-7 dias'
        WHEN dias_restantes >= 8 THEN '8+ dias' END AS
    categoria,
    COUNT(*) AS qtd
FROM em_andamento GROUP BY grupo, categoria ORDER BY grupo,
    categoria;

```

14.3 D.3 relatorios/03_controle_reservas.sql

```
-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: relatorios/03_controle_reservas.sql
-- Grupo: Gustavo Oliveira Pessanha (122051824), Andre Vinicius
--         Giron (122050404)
-- Relatorio gerencial (c) – Controle de reservas (status ativo).
-- =====

\pset border 2
\pset null '(null)'
\set ON_ERROR_STOP on

-- Visao 1: grupo x patio retirada x cidade origem do cliente
SELECT gr.nome_grupo_normalizado AS grupo,
       pa.apelido                AS patio_retirada,
       cl.cidade_origem AS cidade_origem_cliente,
       COUNT(*)              AS qtd_reservas
FROM dw.fato_reserva fres
JOIN dw.dim_grupo    gr ON gr.sk_grupo = fres.sk_grupo
JOIN dw.dim_patio    pa ON pa.sk_patio = fres.sk_patio_retirada
JOIN dw.dim_cliente  cl ON cl.sk_cliente = fres.sk_cliente
WHERE fres.status_reserva IN
      ('CONFIRMADA', 'EM_FILA_ESPERA', 'CONCRETIZADA')
GROUP BY gr.nome_grupo_normalizado, pa.apelido, cl.cidade_origem
ORDER BY gr.nome_grupo_normalizado, pa.apelido, cl.cidade_origem;

-- Visao 2: grupo x horizonte temporal (semana_proxima /
--         mes_proximo / posterior)
WITH base AS (
    SELECT gr.nome_grupo_normalizado AS grupo,
           (tr.data_completa - td.data_completa) AS dias_antecedencia
    FROM dw.fato_reserva fres
    JOIN dw.dim_grupo gr ON gr.sk_grupo = fres.sk_grupo
    JOIN dw.dim_tempo td ON td.sk_tempo = fres.sk_tempo_reserva
    JOIN dw.dim_tempo tr ON tr.sk_tempo =
        fres.sk_tempo_retirada_prevista
    WHERE fres.status_reserva IN
          ('CONFIRMADA', 'EM_FILA_ESPERA', 'CONCRETIZADA'))
SELECT grupo,
       CASE WHEN dias_antecedencia <= 7 THEN
           'semana_proxima'
           WHEN dias_antecedencia BETWEEN 8 AND 30 THEN
           'mes_proximo'
           WHEN dias_antecedencia > 30 THEN 'posterior'
```

```

        ELSE 'indefinido' END AS horizonte,
COUNT(*) AS qtd
FROM base GROUP BY grupo, horizonte ORDER BY grupo, horizonte;

```

14.4 D.4 relatorios/

04_grupos_mais_alugados.sql

```

-- =====
--  Avaliacao 02 – Modelagem de Data Warehouse – Parte II
--  Arquivo: relatorios/04_grupos_mais_alugados.sql
--  Grupo: Gustavo Oliveira Pessanha (122051824), Andre Vinicius
--         Giron (122050404)
--  Relatório gerencial (d) – Grupos mais alugados x cidade do
--         cliente.
-- =====

\pset border 2
\pset null '(null)'
\set ON_ERROR_STOP on

-- Visao 1: ranking de grupos por quantidade de locacoes
SELECT gr.nome_grupo_normalizado AS grupo,
       COUNT(*) AS qtd_locacoes,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC) AS posicao
FROM dw.fato_locacao floc
JOIN dw.dim_grupo gr ON gr.sk_grupo = floc.sk_grupo
GROUP BY gr.nome_grupo_normalizado
ORDER BY qtd_locacoes DESC, grupo;

-- Visao 2: grupo x cidade origem
SELECT gr.nome_grupo_normalizado AS grupo,
       cl.cidade_origem, COUNT(*) AS qtd_locacoes
FROM dw.fato_locacao floc
JOIN dw.dim_grupo gr ON gr.sk_grupo = floc.sk_grupo
JOIN dw.dim_cliente cl ON cl.sk_cliente = floc.sk_cliente
GROUP BY gr.nome_grupo_normalizado, cl.cidade_origem
ORDER BY qtd_locacoes DESC, grupo, cidade_origem;

-- Visao 3: top grupo por cidade
WITH base AS (
    SELECT cl.cidade_origem, gr.nome_grupo_normalizado AS grupo,
           COUNT(*) AS qtd_locacoes,
           ROW_NUMBER() OVER (PARTITION BY cl.cidade_origem

```

```

ORDER BY COUNT(*) DESC,
          gr.nome_grupo_normalizado) AS
rn
FROM dw.fato_locacao floc
JOIN dw.dim_grupo gr ON gr.sk_grupo = floc.sk_grupo
JOIN dw.dim_cliente cl ON cl.sk_cliente = floc.sk_cliente
GROUP BY cl.cidade_origem, gr.nome_grupo_normalizado)
SELECT cidade_origem, grupo, qtd_locacoes
FROM base WHERE rn = 1
ORDER BY qtd_locacoes DESC, cidade_origem;

```

14.5 D.5 relatorios/05_matriz_markov.sql

```

-- =====
-- Avaliacao 02 – Modelagem de Data Warehouse – Parte II
-- Arquivo: relatorios/05_matriz_markov.sql
-- Grupo: Gustavo Oliveira Pessanha (122051824), Andre Vinicius
--        Giron (122050404)
-- Matriz estocastica 6x6 de movimentacao da frota entre patios.
-- =====

\pset border 2
\pset null '(null)'
\pset numericlocale off
\set ON_ERROR_STOP on

-- Forma LONG: par (retirada, devolucao) -> qtd e probabilidade
WITH locacoes_concluidas AS (
    SELECT pr.apelido AS patio_retirada, pd.apelido AS
           patio_devolucao
    FROM dw.fato_locacao floc
    JOIN dw.dim_patio pr ON pr.sk_patio = floc.sk_patio_retirada
    JOIN dw.dim_patio pd ON pd.sk_patio = floc.sk_patio_devolucao
    WHERE floc.status_locacao = 'CONCLUIDA'
           AND floc.sk_patio_devolucao IS NOT NULL),
agregado AS (
    SELECT patio_retirada, patio_devolucao, COUNT(*) AS
           qtd_locacoes
    FROM locacoes_concluidas
    GROUP BY patio_retirada, patio_devolucao)
SELECT patio_retirada, patio_devolucao, qtd_locacoes,
       ROUND(qtd_locacoes::NUMERIC
             / SUM(qtd_locacoes) OVER (PARTITION BY patio_retirada),
             4)::NUMERIC(7,4) AS probabilidade
FROM agregado

```

```
ORDER BY patio_retirada, patio_devolucao;
```

```
-- Forma WIDE: matriz 6x6 (probabilidades) – usa COUNT(*) FILTER
WITH locacoes_concluidas AS (
    SELECT pr.apelido AS patio_retirada, pd.apelido AS
        patio_devolucao
    FROM dw.fato_locacao floc
    JOIN dw.dim_patio pr ON pr.sk_patio = floc.sk_patio_retirada
    JOIN dw.dim_patio pd ON pd.sk_patio = floc.sk_patio_devolucao
    WHERE floc.status_locacao = 'CONCLUIDA'
        AND floc.sk_patio_devolucao IS NOT NULL),
totais_por_retirada AS (
    SELECT patio_retirada, COUNT(*)::NUMERIC AS total
    FROM locacoes_concluidas GROUP BY patio_retirada)
SELECT lc.patio_retirada,
    ROUND(COUNT(*) FILTER (WHERE lc.patio_devolucao = 'Galeao') /
        t.total, 4)::NUMERIC(7,4) AS galeao,
    ROUND(COUNT(*) FILTER (WHERE lc.patio_devolucao = 'Santos
        Dumont') / t.total, 4)::NUMERIC(7,4) AS santos_dumont,
    ROUND(COUNT(*) FILTER (WHERE lc.patio_devolucao =
        'Rodoviaria') / t.total, 4)::NUMERIC(7,4) AS rodoviaria,
    ROUND(COUNT(*) FILTER (WHERE lc.patio_devolucao = 'Rio
        Sul') / t.total, 4)::NUMERIC(7,4) AS rio_sul,
    ROUND(COUNT(*) FILTER (WHERE lc.patio_devolucao = 'Nova
        America') / t.total, 4)::NUMERIC(7,4) AS nova_america,
    ROUND(COUNT(*) FILTER (WHERE lc.patio_devolucao = 'Barra') /
        t.total, 4)::NUMERIC(7,4) AS barra
FROM locacoes_concluidas lc
JOIN totais_por_retirada t ON t.patio_retirada = lc.patio_retirada
GROUP BY lc.patio_retirada, t.total
ORDER BY lc.patio_retirada;
```

```
-- Validacao: soma por linha deve ser 1.0 (tolerancia 1e-6)
```

```
WITH locacoes_concluidas AS (
    SELECT pr.apelido AS patio_retirada, pd.apelido AS
        patio_devolucao
    FROM dw.fato_locacao floc
    JOIN dw.dim_patio pr ON pr.sk_patio = floc.sk_patio_retirada
    JOIN dw.dim_patio pd ON pd.sk_patio = floc.sk_patio_devolucao
    WHERE floc.status_locacao = 'CONCLUIDA'
        AND floc.sk_patio_devolucao IS NOT NULL),
agregado AS (
    SELECT patio_retirada, patio_devolucao, COUNT(*) AS
        qtd_locacoes
    FROM locacoes_concluidas GROUP BY patio_retirada,
        patio_devolucao),
probabilidades AS (
    SELECT patio_retirada,
```

```

        qtd_locacoes::NUMERIC
        / SUM(qtd_locacoes) OVER (PARTITION BY patio_retirada)
        AS p
    FROM agregado)
SELECT patio_retirada, SUM(p) AS soma_prob, ABS(SUM(p) - 1.0) AS
    delta_abs
FROM probabilidades GROUP BY patio_retirada
HAVING ABS(SUM(p) - 1.0) > 0.000001
ORDER BY patio_retirada;
-- (Linha vazia se a matriz e estocastica valida.)

```

Fim do PDF 2. O modelo dimensional, dicionário de dados e DDL completo do DW estão no PDF 1 (docs/relatorio-dimensional.pdf).